

杭州电子科技大学

硕士学位论文

题 目: 面向深度学习训练场景的
内存管理方法研究

研究生 朱颖

专 业 计算机技术

指导教师 赵乃良 教授

完成日期 2024 年 3 月

杭州电子科技大学硕士学位论文

面向深度学习训练场景的
内存管理方法研究

研究生： 朱 颖

指导教师： 赵乃良 教授

2024 年 3 月

**Dissertation Submitted to Hangzhou Dianzi University
for the Degree of Master**

**Research on Memory Management
Methods for Deep Learning Training
Scenarios**

Candidate: Zhu Ying

Supervisor: Prof. Zhao Nailiang

March, 2024

摘要

近年来,深度学习神经网络在多个领域得到广泛应用,数据集和神经网络模型的规模也随之快速增长,然而,用于训练这些神经网络的设备内存容量则发展相对缓慢。在这一背景下,如何高效地利用有限内存空间进行深度学习训练,成为当下深度学习技术发展中亟需解决的重要问题。现有的内存优化技术如重计算、内存交换及其组合的自适应内存优化方法,通过管理训练数据的分配和释放时机,从而降低训练所需的内存量。同时,现有的内存分配算法则致力于在训练数据的分配和释放过程中优化内存空间布局,以减少内存浪费。但是当前方法未考虑到模型结构差异,以及模型的迭代训练等情况,目前仍然存在以下问题:

(1) 模型结构差异导致自适应内存优化方法执行效率低下问题。现有的自适应内存优化方法并未充分考虑模型的结构信息,忽视了神经网络中不同训练数据的重计算成本和内存交换成本的差异,以及优化后的数据分配和释放时机对训练效率的影响。在进行内存优化的同时降低了内存利用效率和训练吞吐量。

(2) 模型迭代训练过程中产生的内存碎片问题。当前的内存分配算法通常仅考虑数据请求大小这一单一特征来管理内存数据。然而在模型多轮迭代训练过程中,数据释放和分配过程并行执行,且包含大量临时数据。使用现有的内存分配算法会导致内存空间中存在大量内存碎片,从而造成内存浪费。

本文针对上述问题,围绕自适应内存优化方法和内存分配算法展开研究,具体工作如下:

(1) 针对模型结构差异导致自适应内存优化方法执行效率低下问题,本文基于重计算和内存交换技术,提出了一种基于双智能体强化学习的自适应内存优化方法, REMEM(Reinforcement Learning based GPU Memory Management Algorithm),该方法为神经网络训练过程中产生的数据搜索合适的内存优化策略以及内存优化时机。首先,该方法构建拟合神经网络训练过程的多维状态矩阵作为强化学习算法的搜索环境,并基于神经网络模型结构确定内存优化策略搜索空间;其次,采用双智能体搜索算法协同搜索内存优化策略的应用数据和执行时间;最后,根据动态反馈算法计算策略搜索结果的奖励值,通过多轮迭代优化以得到收益最大的策略搜索结果,在达到计算加速设备内存优化目标的同时,避免密集化的通信和计算资源消耗。经实验验证,使用 REMEM 方法搜索的策略对神经网络模型进行内存优化后,不仅能显著提高设备上模型训练最大

batch size, 同时相对于 Capuchin、HOME 等自适应内存优化方法, 实现了最高 32.7%和 9.18%训练吞吐量提升。

(2) 围绕模型迭代训练过程中产生的内存碎片问题, 本文提出了一种基于 AI 训练任务访存特征的内存分配算法, MARSP(Memory Allocator based on Request Size and Period), 该方法依据 AI 训练任务中的数据请求访存特征进行内存空间的高效管理和分配。首先, MARSP 方法提出 AI 训练任务内存请求特征分析算法, 该算法结合模型训练特征动态识别并分析内存请求访存周期及周期内请求特征; 其次, 设置多级数据管理结构对内存空间进行分区, 以存放具有不同访存特征的数据; 最后, 采用分时数据管理方法和双端数据分配方法来管理训练任务中的数据访存请求, 从而降低数据分配过程中带来的内存浪费。经实验验证, MARSP 相较于 TensorFlow 和 PyTorch 以及 PagedAttention 中的内存分配算法, 有效缓解了分配过程中产生的内存碎片问题, 显著减少了模型训练过程中的内存需求量。

关键词: 深度学习, GPU 内存管理, 自适应内存优化, 内存分配

ABSTRACT

In recent years, deep learning neural networks have been widely applied across various domains, leading to a rapid increase in the scale of datasets and neural network models. However, the memory capacity of training devices has progressed relatively slowly. Against this backdrop, the efficient utilization of limited memory space for deep learning training has become a crucial issue. Existing memory optimization techniques, such as rematerialization, memory swapping, and adaptive memory optimization methods that combine these approaches aim to reduce the amount of memory required for training by managing the allocation and deallocation timing of training data. Meanwhile, existing memory allocation algorithms strive to optimize the memory space layout during the allocation and deallocation processes of training data to minimize memory waste. However, existing methods fail to account for differences in model structures and the cyclic iterative access patterns of memory requests during the training process. Consequently, leading to the following problems:

(1) The inefficiency execution of adaptive memory optimization methods due to differences in model structures. Existing adaptive memory optimization methods inadequately consider structural information of the models, neglecting the differences in rematerialization costs and memory swapping costs across different layers within neural networks, as well as the influence of the timing of memory optimization method execution on training efficiency. Reducing the memory utilization efficiency and the training throughput while conducting memory optimization.

(2) The memory fragmentation caused by iterative model training. Current memory allocation algorithms typically consider only the size of data requests, managing memory data through the division into differently sized blocks. However, during the model training process, data requests exhibit characteristics of periodic iterative access. Using existing memory allocation algorithms results in a amount of memory fragmentation within the memory space, leading to memory wastage.

Addressing these issues, this thesis focuses on adaptive memory optimization methods and memory allocation algorithms, with contributions as follows:

(1) To tackle the problem of inefficient memory optimization, this thesis introduces the REMEM(Reinforcement learning based GPU Memory Management

Algorithm) method based on recomputation and memory swapping techniques, combined with reinforcement learning. This method searches for suitable memory optimization strategies and optimization timing for feature mapping data generated during neural network training. Firstly, it defines a memory optimization strategy space based on the neural network model structure and constructs a multi-dimensional state matrix to fit the neural network training process as the training ground for the reinforcement learning algorithm. Secondly, it employs a dual-agent reinforcement learning method to collaboratively search for the application data and execution timing of memory optimization strategies. Finally, it iterates over multiple rounds of optimization based on the reward values of strategy search results to achieve the most beneficial strategy search outcome. This approach not only reduces GPU memory consumption during model training but also improves throughput. Experimental results demonstrate that the strategies searched using REMEM significantly increase the maximum batch size for model training on devices and achieve up to 32.7% and 9.18% improvements in training throughput compared to existing adaptive memory optimization methods such as Capuchin and HOME.

(2) Addressing the problem of memory fragmentation generated during model training, this thesis proposes a memory allocation algorithm based on AI training task access features, termed MARSP (Memory Allocator based on Request Size and Period). MARSP efficiently manages and allocates memory space based on the data request access features in AI training tasks. Firstly, the MARSP method proposes an algorithm for analyzing memory request features in AI training tasks, which dynamically identifies and analyzes features of memory request based on model training characteristics. Secondly, it sets up a multi-level data management structure to partition memory space for storing data with different access features. Lastly, it adopts a time-sharing data management method and a dual-ended data allocation method to manage data access requests in training tasks, thereby reducing memory wastage caused by the data allocation process. Experimental validation shows that MARSP effectively alleviates memory fragmentation issues during allocation compared to memory allocation algorithms in TensorFlow and PyTorch, significantly reducing memory requirements during the model training process.

Keywords: Deep Learning, GPU Memory Management, Adaptive Memory Optimization, Memory Allocator

目 录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 内存优化技术.....	2
1.2.2 自适应内存优化方法.....	4
1.2.3 内存分配算法.....	5
1.2.4 国内外研究现状总结.....	7
1.3 本文研究内容.....	7
1.4 本文组织结构.....	8
第二章 相关理论基础	10
2.1 深度学习模型训练方法.....	10
2.2 内存优化技术.....	11
2.2.1 重计算技术.....	11
2.2.2 内存交换技术.....	12
2.3 自适应内存优化方法.....	13
2.4 内存分配算法.....	19
2.5 强化学习算法.....	22
2.6 本章小结.....	23
第三章 基于双智能体强化学习的自适应内存优化方法	24
3.1 问题描述.....	24
3.2 REMEM 方法概述	26
3.3 拟合模型结构的自适应内存优化问题建模方法	27
3.3.1 基于 Profiling 的模型信息收集	27
3.3.2 模型训练状态建模.....	28
3.3.3 内存优化策略搜索空间建模.....	30
3.4 基于双智能体强化学习的内存优化策略搜索方法	31
3.4.1 双智能体策略搜索算法.....	31
3.4.2 策略模拟执行算法.....	32
3.4.3 策略反馈优化算法.....	34
3.5 REMEM 方法实现	36
3.6 本章小结.....	38
第四章 基于 AI 训练任务访存特征的内存分配算法	39
4.1 问题描述.....	39
4.2 MARSP 方法概述	41

4.3 针对 AI 训练任务的访存请求特征分析算法	41
4.3.1 访存请求周期识别算法	42
4.3.2 周期内请求特征分析算法	44
4.4 多级分区的内存数据管理	45
4.5 分时双端内存分配管理方法	46
4.5.1 分时数据管理方法	46
4.5.2 双端数据分配方法	49
4.6 本章小结	50
第五章 实验	51
5.1 REMEM 有效性实验分析	51
5.1.1 实验设置	51
5.1.2 模型训练 batch size 分析	52
5.1.3 模型训练吞吐量分析	54
5.1.4 双智能体搜索算法有效性分析	56
5.2 MARSP 有效性实验分析	57
5.2.1 实验设置	57
5.2.2 训练最大内存需求量分析	58
5.2.3 内存碎片分析	61
5.2.4 分时数据管理方法有效性分析	61
5.2.5 双端数据分配方法有效性分析	63
5.3 本章小结	64
第六章 总结与展望	65
6.1 本文工作总结	65
6.2 未来展望	66
参考文献	67

第一章 绪论

1.1 研究背景与意义

深度学习作为人工智能领域的重要分支，近年已在自然语言处理^[1,2]、图像识别^[3,4]等多个关键技术领域均表现出非凡的应用价值。这吸引了越来越多的技术巨头投入大量人力和财力资源，推动这些模型持续向更复杂、更智能的方向演进。为了更好地捕捉数据中的复杂模式和关联性^[5]，神经网络模型的深度和广度不断增加。

然而，训练加速设备的内存资源却是限制模型的训练和应用的一个关键因素^[6,7]。在模型训练过程中，加速训练设备上需要初始化参数数据，平均每250M的参数需要占用1GB加速设备内存，剩余的空间则用来存储训练过程中的中间数据，如特征数据，梯度数据，临时卷积空间等等，这些数据的尺寸与样本的训练 batch size 成线性比例关系。受设备的有限内存资源限制，单个加速训练设备上通常难以应用较大的模型来提取数据信息，或者只能使用小 batch size 的样本数据来进行训练，这会延长模型训练时间，降低模型训练吞吐量，同时也会影响模型预测的准确度。即使可以使用数据并行^[8]、模型并行^[9]或流水线并行^[10]等技术^[11,12]在多个加速设备上进行分布式训练，从逻辑上扩大训练设备的内存资源，然而在分布式训练过程中，各个设备上通常需要存放一些冗余的训练数据，除此之外，设备之间的数据同步过程会带来额外的时间消耗，延长训练时间。近年来，训练设备生产厂商也观察到该问题，并不断对设备的内存资源进行升级^[13]，以 GPU 设备为例，从 P100 到最新的 H200，10 年内 GPU 设备的内存大小增长了 10 倍左右（12GB-141GB），然而同期模型规模则增长了千倍，如 ResNet-50（340M）^[14]到 PLaM2（330B）^[15]，训练数据集的尺寸也随之飞速增长^[16]，如 6 万张图像的 MNIST^[17]到 30 亿张图像的 JFT-3B^[18]。训练设备的内存资源与模型训练过程中的内存需求的增长速度呈现明显不平衡。因此如何对模型训练过程中的内存进行管理，从而更高效利用单个训练加速设备的内存资源，是一个具有重要研究意义和实际价值的问题。

针对模型训练过程中的内存管理问题，模型量化^[19]、剪枝^[20]等方法能有效减少对设备内存的需求，但是这些方法都会在一定程度上降低模型训练的精度，从而影响模型预测的准确性。对训练过程中的数据进行压缩存储^[21]也是常用的内存优化方法之一，然而适用于压缩技术的数据需要具有高度稀疏性的特征，

这限制了压缩技术的应用范围。研究人员观察到模型训练过程中的特征映射数据具有较大的优化价值，提出了重计算^[22]和内存交换^[23, 24]方法对特征映射数据的分配和释放时机进行优化，但是重计算和内存交换方法在节约内存的同时也带来了额外的计算和通信。鉴于上述内存优化方法都具有局限性，为了进一步提高内存优化效果，学者对于内存优化策略进行了自适应组合^[25, 26]，然而，现有的组合策略未充分考虑模型结构对优化效果的影响，很难在节约内存空间的同时兼顾模型训练效率。除了对于训练数据进行管理与优化，使用内存分配算法管理训练数据的分配和释放请求，能够优化内存空间布局，实现内存资源的高效复用。主流 AI 训练框架如 PyTorch^[27]，TensorFlow^[28]中都实现了自定义的内存分配算法，研究学者也提出了分区管理内存空间^[29]以及按页分配训练数据内存^[30]等方法，但是这些方法没有考虑到模型训练过程中数据访存访问具有周期性这一重要特征，仍然面临因为内存碎片造成的资源浪费问题。

为了解决上述问题，本文面向神经网络模型的训练场景中，如何实现对计算加速设备的高效内存管理展开研究。针对模型结构差异导致自适应内存优化方法执行效率低下问题，研究基于双智能体强化学习的自适应内存优化方法；针对模型迭代训练过程中产生的内存碎片问题，研究基于 AI 训练任务访存特征的内存分配算法。最后，通过性能对比实验，验证本文方法的有效性。

1.2 国内外研究现状

随着深度学习的不断应用与发展，对于训练设备的内存资源需求也日益增长，如何减少训练过程中的内存资源消耗，高效利用有限的内存资源是一个重要的问题。为此，本文面向深度学习训练过程中的内存优化技术、自适应内存优化方法、以及内存分配算法等方面进行研究。其中主要涉及重计算技术、内存交换技术、自适应内存优化方法、内存分配算法等技术。接下来本文对上述技术进行研究与分析。

1.2.1 内存优化技术

受硬件资源的限制，训练设备内存不足往往成为模型训练的瓶颈，这制约了训练的规模和性能，开发有效的内存优化技术对于降低训练内存消耗至关重要。通过对神经网络模型训练过程中的数据进行分析可以找到模型训练过程内存优化的抓手。在模型训练的过程中，首先会在训练设备如 GPU 中初始化模型的权重参数数据；其次，训练数据从 CPU 上拷贝到 GPU 中，并在前向传播

过程中经过一系列特征提取层和分类层的处理，得到神经网络的预测输出值，在这期间会生成特征映射数据（又称激活数据），以及一些卷积算子计算过程中所需的临时变量数据；接着，利用预测输出值与实际标签值之间的差异，计算损失函数^[31]的值，即模型的优化目标；然后，在反向传播过程中计算损失函数对模型参数的梯度数据，最后，利用梯度下降^[32]等优化方法来更新网络的权重，以最小化损失函数。通过迭代执行前向传播、损失计算和反向传播的过程，不断调整网络参数，提高模型的预测精度。

观察模型训练过程中的数据生成情况可知，主要有：参数数据、特征映射数据、临时变量数据和梯度数据和优化器状态数据。其中，临时变量数据、梯度数据和优化器状态数据在生成后很快会被使用并释放，因此模型训练过程中的内存瓶颈时期一般在正向传播结束，反向传播刚刚开始阶段，此时设备内存中包含了模型参数数据，所有的特征映射数据，并且需要分配新的内存来存储梯度数据。所以针对模型训练过程中的内存使用瓶颈，业界的内存优化通常集中在长时间驻留内存的参数数据和特征映射数据间。

对于参数数据的内存优化主要包括模型压缩技术和分布式并行训练技术。模型压缩技术主要包括模型量化（Model Quantification）和模型剪枝（Model Pruning）两种方法。模型量化^[33]通过降低参数的精度（例如，从 32 位浮点数减少到 8 位或更低），有效减小模型的存储需求和加快计算速度，这在移动设备和边缘计算中尤为重要。模型剪枝^[20]则通过移除神经网络中的非关键权重，减少模型的复杂度和内存占用。但是模型压缩技术在优化内存占用的同时会影响模型预测的准确度，且该方法主要应用于模型的推理和部署领域^[21]。另一方面，分布式并行训练技术，包括模型并行^[9]和 ZeRO^[34, 35]改进的数据并行技术，在处理大规模模型时显得尤为关键。模型并行将大型模型分割并分布在不同的计算节点上，单个计算节点所需要存储的模型参数数据量也随之减少。而 ZeRO 优化技术则改进了传统的数据并行方法，通过分区算法将模型的参数、梯度和优化器状态等分布到不同的训练设备上，单个训练设备上不再需要保存模型的完整副本，有效减少了冗余数据带来的内存浪费。分布式并行技术显著提高大规模模型训练的效率和可扩展性，然而这些技术也增加了通信开销和编程复杂性，尤其是在涉及到多个处理单元之间数据同步过程时。

对于特征映射数据的优化方法主要方法有数据压缩（Data Compression）、重计算（Rematerialization）和内存交换（Memory Swapping）技术。数据压缩^[36]技术通过减少数据表示的位数来降低内存占用。近年来，相关研究^[37]提出了深度神经网络的低精度训练和混合精度训练，减少了数据存储和计算的资源消耗，然而，这种方法可能会导致精度损失，尤其在某些对精度敏感的任务中，

这种损失可能会对模型性能产生显著影响。重计算技术^[22, 38]在前向传播过程中释放部分激活数据，并在反向传播过程中重计算这些数据，这种方法减少了内存消耗，但增加了额外的计算负担。内存交换^[24]技术则通过在正向传播期间将激活值转移到 CPU 内存中，然后在相应的反向计算时将其预取回 GPU 内存，释放该段时间中数据在 GPU 上内存占用，以节省 GPU 内存。这种方法的挑战在于需要优化数据的转移时机，以减少 CPU 和 GPU 之间通信的延迟。比如 vDNN 方法^[39]采用启发式策略对 CNN 网络进行优化，该方法试图将数据的传输过程与卷积层的计算重叠，但是这种方法基于经验假设，忽略了不同卷积层的计算耗时也存在差异这一问题，且该方法的适用性在通用 DNN 中受限。moDNN^[40]、AutoTM^[41]和 SwapAdvisor^[42]等研究则通过分析特征数据的生命周期和优先级来决定交换时机，这提高内存交换技术的优化效率，但是这些方法仍然受到 CPU 和 GPU 的总线通信效率的约束。

综上所述，虽然这些优化技术在减少内存消耗方面有显著作用，但它们各自也面临局限性，例如可能导致模型精度的降低、计算与通信成本的增加等问题。因此，这些内存优化技术的普适性受到一定程度的影响。

1.2.2 自适应内存优化方法

为了扩大内存优化技术的应用范围，提高模型训练过程中的内存管理效率。研究学者提出了自适应内存优化方法，这类方法通过组合多种内存优化技术，并根据具体模型的结构和需求进行动态调整，以实现更高效的内存管理和计算性能。

cDMA^[36]的研究有效结合了数据压缩和内存交换这两个内存优化技术。cDMA 利用了 ReLU 激活函数产生的输出层数据的稀疏性。通过设计一个专门的硬件核心集成于 GPU 之中，cDMA 实现了这些数据的高效压缩，旨在减少在 GPU 和其他存储设备间传输的数据量，进而提升整体性能。然而，这种方法将压缩算法固化于硬件之中，导致其应用相对单一，缺乏灵活性。另一方面，CSWAP^[43]方法则进一步拓展了对 ReLU 输出数据的稀疏性和稀疏可变性（即随着训练进程，不同层的稀疏程度会发生变化）的利用。CSWAP 提出了一种选择性压缩架构，它在数据传输过程中根据压缩和传输性能的综合评估来决定是否执行压缩。为实现这一目标，CSWAP 采用了两个机器学习模型对关键指标进行预测，从而能够准确地选择适合进行压缩加传输的层，而对其他层仅执行数据传输。通过这种动态和自适应的方法，CSWAP 在优化了显存使用的同时，保持了存储和计算效率。

SuperNeurons^[44]是首个将重计算和内存交换技术结合应用的内存优化方法，该方法采用启发式策略，针对不同类型的神经网络层实施特定的优化手段：使用内存交换技术优化卷积层数据，而采用重计算技术优化计算时间较短但占用内存较多的层，如池化层。另一方面，Capuchin 方法^[45]则通过实时跟踪动态张量访问模式来制定内存管理决策，基于贪心算法对张量访问模式和存储器的潜在优化空间进行分析，从而提供对内存优化技术执行时机及方式的细粒度和灵活控制。然而，这些方法在考虑模型信息时存在局限性。例如，SuperNeurons 主要针对特定类型的深度神经网络（DNN）层进行张量放置优化，而 Capuchin 依赖于从第一个 DNN 层到当前层的数据流分析来确定策略。鉴于重计算和内存交换技术虽目标一致——减少内存消耗，但依赖于不同的技术途径并通过独立的资源实现，研究人员提出了通过动态规划方法^[25]寻找结合重计算和内存交换的最佳优化策略放置序列。特别地，POET 方法^[46]为嵌入式设备训练场景设计了基于混合整数线性规划^[47]（MILP）的自适应内存优化方法，结合重计算和内存优化技术，在给定的内存预算和运行时间约束下，寻找能量消耗最少的训练策略。HOME^[48]则运用定制的粒子群优化算法^[49]来全局优化 DNN 模型的每个特征数据的放置，显著降低了搜索空间。这种对整个模型信息的全局感知使得 HOME 能够在限定的 GPU 内存约束下实现高效性能。然而，这些方法主要关注于数据应用策略的选择上，尚未充分考虑内存优化策略执行时，数据释放和分配时机对内存优化和训练效率的影响。

值得注意的是，自适应内存优化方法的关键在于如何准确评估和预测不同内存优化方法在特定网络结构中的效益和代价。这需要深入分析神经网络的结构特性，如层间依赖关系、数据流动模式等。当前的研究正在探索更加高级的算法，比如基于机器学习的方法^[50]，来自动识别并实施最佳的内存优化策略。然而，这一领域仍面临许多挑战，如怎样平衡计算开销和内存优化的效益，以及在不同类型和规模的网络中如何广泛应用这些技术。因此，模型训练过程中的自适应内存优化仍是一个在迅速发展的研究方向。

1.2.3 内存分配算法

为了解决 GPU 在深度学习和高性能计算领域时面临的内存管理挑战，减少频繁内存分配和释放操作带来的内存碎片，学者提出通过维持一个单独的内存分配器的方式管理数据访存请求。依赖训练设备系统进行内存管理方式在应对频繁的内存分配和释放请求时效率低下，容易造成内存浪费，分配器通过一个预先分配的内存块集合集中管理设备内存空间，这种方法不仅提高了内存分配

的效率，还增强了对动态内存请求的适应能力，从而在执行计算任务时优化设备内存资源利用率。

因此对于 GPU 内存分配管理方法受到了广泛关注，很多研究人员针对 AI 模型训练过程中的访存特征提出了专门的内存分配算法，例如，ShortcutFusion^[51]方法为 CNN 模型中的残差模块分配了支持快速重用的静态内存空间，而 PagedAttention 方法^[30]则受虚拟内存和分页概念启发，通过分配多个不连续的物理内存块来存储大语言模型训练过程中生成的键值向量，减少了模型训练过程中的内存需求。然而，这些内存分配算法的有效性受到了模型结构的限制。

同时，NVIDIA、Google、Facebook 等顶尖的科技公司 and 学术机构，也在积极探索和发展这方面的技术。这些研究主要集中于如何更高效地管理模型训练过程中的内存资源，以提高内存利用率和计算性能，同时减少内存碎片和延迟。例如，NVIDIA 在 CUDA11 及更高版本中提供了基于流排序的异步内存分配器^[52]，它异步分配器允许用户按流顺序分配和释放数据。分配器可以自由地重新分配内存，只要它可以保证兼容的内存访问不会在时间上重叠。在 Google 的 TensorFlow 深度学习框架中，内存分配算法是以 Dlmalloc^[53]中的伙伴系统^[54]为基础实现的 BFC (Best Fit with Coalescing) 分配算法。BFC 算法维护一个可用内存块的列表，当请求内存时，它会尝试找到同请求大小相近的空闲内存块，并通过合并相邻的空闲内存块和拆分大块内存来满足小块内存请求，从而进一步减少内存碎片。这种方法旨在平衡内存利用率和分配速度，使得 TensorFlow 能够有效地处理模型训练过程中的内存数据请求。由 Facebook 人工智能学院开发的 PyTorch 中则采用了不同的策略，主要通过一个基于 slab 思想^[55]的缓存分配器来管理 GPU 内存。这个缓存分配器工作方式类似于内存池，当张量被释放时，它们占用的内存不会立即返还给 GPU，而是保留在一个缓存中。当新的内存请求产生时，分配器首先检查缓存中是否有足够的空间来满足这个请求，如果有，就直接使用缓存中的内存，而不是向 GPU 请求新的内存。这种方法减少了频繁的内存分配和释放操作，从而优化了内存使用效率和减少了碎片化问题。但是，目前的这些内存分配算法通常只关注内存分配请求的大小，而未考虑模型训练过程中内存分配和释放请求具有周期性这一重要特征。

总体而言，针对模型训练过程中的内存分配算法是一个活跃且不断发展的研究领域，国内外的研究学者们都在不断推动这一技术向更高效、更智能的方向发展。尽管如此，这一领域仍面临着许多挑战，例如，内存分配算法如何适应不同类型和规模的计算任务，以及如何在内存利用率和计算性能之间寻求平衡等等问题仍然需要解决。

1.2.4 国内外研究现状总结

针对国内外研究现状，本文有如下总结：

(1) 现有的内存优化技术主要通过改变数据尺寸或者调整数据分配和释放时机，以减少模型训练过程中的内存占用，然而这些方法普遍存在影响精度或者因为通信和计算成本增加而影响训练效率的问题，因此在应用范围上存在显著的局限性。为了应对这一挑战，现有的自适应内存优化方法通过结合多种内存优化技术来拓展应用范围，但是仍未深入考虑不同模型结构在训练过程中的数据访问模式差异，以及内存优化时机对模型训练过程产生的具体影响，从而出现内存优化效率低下的问题。

(2) 现有的内存分配算法虽然在处理内存请求时考虑了数据的尺寸，以优化内存利用率和减少分配延迟，但大多数算法并未充分考虑深度学习训练任务的访存特征。在深度学习的训练中，使用相同的模型结构对不同的数据进行迭代训练，因此数据的分配和释放请求呈现出明显的周期性。但现有内存分配算法往往仅侧重于当前的访存请求，这种缺乏预见性的内存管理方式会导致大量内存碎片的产生，降低了内存利用率。

1.3 本文研究内容

为了解决深度学习训练场景下的内存优化方法执行效率低下、以及内存碎片导致的内存浪费的问题，本文围绕自适应内存优化方法、以及内存分配算法进行研究，提出了基于强化学习的 REMEM 方法，为模型训练过程中产生的数据选择内存优化策略，适时生成和释放训练数据。在接收数据的分配和释放请求时，采用 MARSP 内存分配算法管理请求对应的内存空间，提高内存利用率。

本论文主要研究内容如下：

(1) 针对模型结构差异导致自适应内存优化方法执行效率低下问题，本文提出基于双智能强化学习的自适应内存优化方法，REMEM。REMEM 方法以重计算和内存交换技术为基础，旨在为模型训练过程中的数据搜索分配合适的内存优化策略和内存优化时机。首先，该方法通过 Profiling 技术收集神经网络模型信息和设备信息，基于这些信息，构建了一个训练状态矩阵，用以模拟神经网络训练过程中的内存资源、计算资源和通信资源的消耗状态；其次，采用两个强化学习智能体 `action_agent` 和 `time_agent` 来协同搜索内存优化策略的最优应用数据点和执行时间；然后，在状态矩阵中动态模拟策略搜索结果在实际环境中执行情况，并根据资源使用情况计算奖励值，从而评估不同策略的效益；

最后，通过多轮迭代优化，以得到收益最大的策略搜索结果，根据策略搜索结果优化计算图中的数据依赖关系，避免资源利用冲突，在达到内存优化目标的同时保持模型训练效率。

(2) 模型迭代训练过程中产生的内存碎片问题，本文提出基于 AI 训练任务访存特征内存分配算法，MARSP。该算法依据 AI 训练任务中的数据请求访存特征对训练设备的内存进行管理。首先，MARSP 方法提出基于 AI 训练任务的访存请求特征分析算法，该算法基于滑动窗口和距离差分方法对模型训练访存请求的规律进行分析；其次，在内存数据管理上，分别采用 Block 和 Chunk 这两种数据管理结构对将内存空间分为小请求数据区，常规请求数据区和临时请求数据区，以存放具有不同访存特征的数据；最后，MARSP 方法采用分时数据管理方法管理多个训练周期的访存请求，使用双端数据分配方法为单个训练周期中的数据请求分配内存空间，在减少内存碎片的同时保留整块连续的内存空间，从而降低数据分配过程中的内存资源浪费。

(3) 本文在 TensorFlow 框架的基础上实现并验证了 REMEM 方法。REMEM 同 TensorFlow 中的内存优化方法，Capuchin 方法和 HOME 方法相比，不仅能减少模型训练过程中 GPU 内存消耗，同时提高了模型训练过程中的吞吐量。同时本文实现了 MARSP 内存分配算法，实验发现，同 PyTorch 和 TensorFlow 框架，以及 PagedAttention 中的内存分配算法相比，MARSP 能有效降低模型训练过程中产生的内存碎片数量，显著降低模型训练过程的内存需求。

1.4 本文组织结构

本文共有六个章节，如图 1.1 所示组织结构如下：

第一章：绪论。介绍本文的研究背景及意义、国内外研究现状、以及本文的研究内容。

第二章：相关理论基础。本章首先分析了深度学习模型的训练方法，其次，分别介绍了重计算技术以及内存交换技术这两个内存优化技术的原理与流程。然后，介绍了基于自适应内存优化方法的相关理论。再次，介绍了常用的内存分配算法的技术细节。最后，本章介绍了常用强化学习策略优化方法的原理。

第三章：基于双智能体强化学习的自适应内存优化方法。本章首先分析了模型中模型结构对内存优化方法应用效率的影响。其次，分别解释了策略搜索器中的拟合模型结构的自适应内存优化问题建模方法，以及双智能体强化学习的优化策略搜索方法。最后，介绍了本章提出的自适应内存优化方法的具体实现流程。

第四章：基于 AI 训练任务数据访存特征的内存分配算法。本章首先分析了深度学习训练场景下的内存分配和释放请求的特征，以及当前内存分配算法在该场景下存在的问题。其次，在方法概述部分介绍本章提出的 MARSP 内存分配算法的具体流程。最后，详细介绍了本文提出的针对 AI 训练任务的内存请求特征分析算法，多级分区内存数据管理，以及分时双端内存分配管理方法。

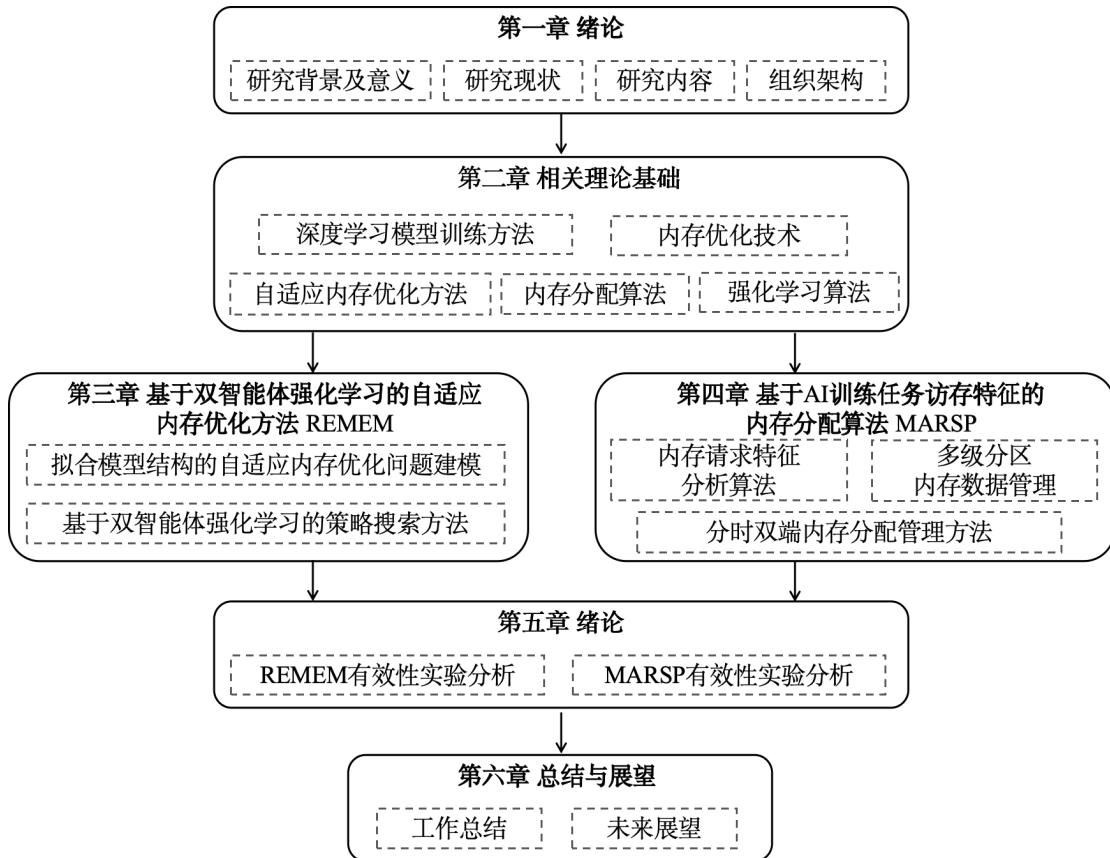


图 1.1 本文组织架构

第五章：实验。本章首先验证 REMEM 方法的有效性，本研究在 TensorFlow 框架的基础上实现了 REMEM 方法，并在五个常用模型上进行了训练验证，实验结果表明，REMEM 对比传统的 TensorFlow 训练方法，显著提升了模型最大训练 batch size；相较于 Capuchin 方法和 HOME 方法，有效提高了模型训练吞吐量。其次，本章实现并验证了 MARSP 内存分配算法的有效性。实验采用多个模型训练过程中的内存请求跟踪日志进行测试，并与 PyTorch 和 TensorFlow 框架以及 PagedAttention 中的内存分配算法进行比较。分别在训练过程中的最大内存需求量、内存碎片、分时数据管理方法对训练内存需求量的影响、以及双端数据分配算法对内存碎片的影响等几个方面进行性能评估实验及分析。

第六章：总结与展望。首先对本文的研究工作进行总结，然后提出未来研究的工作展望。

第二章 相关理论基础

2.1 深度学习模型训练方法

本文面向深度学习模型训练过程中内存管理领域进行研究，需要对深度学习模型的训练过程以及训练中的内存使用情况进行深入分析。深度学习模型的训练过程主要包括数据的加载、前向传播、损失函数的计算、反向传播以及参数的迭代优化。下文将以一个包含输入层、若干隐藏层以及一个输出层的前馈神经网络为例，介绍模型的训练过程。

首先，训练开始前，在计算加速设备（如 GPU）中初始化模型中的参数数据，即为网络中各层的权重 ω_l 和偏置 b_l 分配内存空间并设置初始值。

其次，在训练开始时，读取训练数据进行正向传播计算过程。训练数据通常被分为多个批次（batches），从本机或者远程文件服务器拷贝至计算加速设备内存中以加速计算过程。每个批次的数据通过模型的各个层进行前向传播，在前向传播的起始，输入层接收原始输入数据 T_0 ，然后，数据通过网络的每一层进行传递和变换以提取数据中的特征信息。如式（2.1）所示，在每一层，通过对前一层的激活值 T_{l-1} 进行加权求和，并加上偏置项 b_l ，然后将线性组合结果 z_l 通过激活函数 σ 进行非线性变换，得到当前层的激活值 $T_l = \sigma(z_l)$ 。这个激活值会被传递到下一层作为输入，直到最后一层（输出层）。

$$z_l = \omega_l * T_{l-1} + b_l \quad (2.1)$$

模型的输出随后被用来计算损失函数^[31]，这是一个衡量模型预测与真实标签差异的指标。损失函数的数学表达式可以概括为 $L(y, \hat{y})$ ，其中 y 是真实标签， $\hat{y}$ 是模型输出的预测值。

然后，为了优化模型参数，通过反向传播算法计算损失函数对每层权重 ω_l 和偏置 b_l 的梯度。这一过程从损失函数开始逆向工作：首先，需要计算损失函数相对于输出层激活 T_L 的梯度，并通过链式法则计算损失函数相对于输出层的线性组合 z_L 的梯度，接着，对于每一层 $l = L-1, L-2, \dots, 2, 1$ ，均需要计算损失函数相对于该层激活输出的梯度 $\theta L / \theta T_l$ 和线性组合 z_l 的梯度 $\theta L / \theta z_l$ 。然后，根据线性组合的梯度数据，可以使用以下公式计算损失函数相对于权重和偏置的梯度。

$$\text{权重梯度: } \theta L / \theta \omega_l = (T_{l-1})^T * \theta L / \theta z_l \quad (2.2)$$

$$\text{偏置梯度: } \theta L / \theta b_l = \theta L / \theta z_l \quad (2.3)$$

最后，计算得到的梯度用于更新模型的参数，这一步骤通常是通过梯度下降算法完成的。参数的更新公式为 $\omega = \omega - \eta * \theta L / \theta \omega$ ，其中 η 是学习率，是一个预先设定的正数，决定了参数更新的步长。在参数更新完毕后，释放模型训练过程中的其他数据，仅保留更新后的参数用于下一轮训练。通过迭代训练过程，模型的参数逐渐调整，减少模型预测与实际结果之间的差异，从而提高模型预测的准确度。

通过观察深度学习模型训练过程可知，在正向传播阶段，会不断生成激活值 T_l ，而在随后的反向传播阶段，随着计算的进行，梯度信息亦被逐步生成。并且在反向传播中的第 l 层的数据计算中，需要借助正向传播中的数据 T_l 和 T_{l-1} 。因此，在常规的模型训练过程中，正向传播过程中生成的激活值会一直存放在计算加速设备内存中，直到对应的梯度数据计算完毕。此外，在每一个训练周期中，会产生大量的特征数据（即激活值）、梯度数据及临时数据，这导致训练过程需要不断地分配及释放相应的内存资源。

2.2 内存优化技术

在深度学习领域，随着模型规模的不断扩大，模型在训练过程中对内存资源的需求也成指数级增长，这常常超出了普通硬件设备的内存容量，内存优化成为了提升模型训练效率和实用性的关键挑战。为应对这一问题，研究人员提出了很多内存优化方法来对模型训练过程中的数据进行管理，以减少模型训练过程中的内存消耗。本文将重点介绍两种常用的轻量化内存优化技术：重计算技术和内存交换技术。

2.2.1 重计算技术

在深度学习模型训练的过程中，尤其是在处理大型网络结构时，内存资源的高效利用变得尤为重要。重计算（Rematerialization）技术应运而生，旨在通过优化内存使用来解决这一问题。该技术的核心思想是在训练过程中减少对前向传播过程中的特征映射数据（激活值）的存储需求。在深度学习模型的训练过程中，网络在前向传播阶段计算每一层的输出，并将这些输出存储下来，以便在反向传播计算梯度数据的过程中使用。重计算技术调整了这一过程，它只选择性地存储某些前向传播关键层的输出数据，而非所有层的输出，这显著减少了内存的占用。然而，这也意味着在反向传播阶段，当需要那些未被存储的

中间数据时，必须重新执行相应的前向传播计算来生成这些数据。一旦这些数据被重新计算出来，模型便可以像往常一样进行梯度的计算和权重的更新。

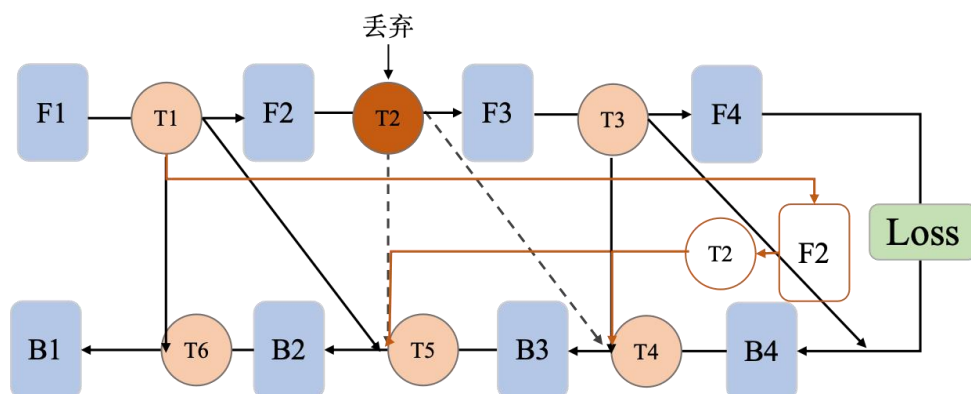


图 2.1 重计算技术示意图

图 2.1 中是一个简单的四层线性网络示意图。其中，F1-F4 表示的是模型的前向传播计算层，其中 T1、T2、T3 是前向传播过程中输出的特征数据，B1-B4 是模型的反向传播计算层，其中 T4、T5、T6 是反向传播过程中计算的梯度数据。在常规模型训练过程中，数据 T1-T3 会保留在 GPU 内存中，因为这些特征数据会在反向传播计算梯度的过程中被重新使用。以特征数据 T2 为例，反向传播过程中的 B2 和 B3 层需要使用该数据，然而 T2 数据从产生到再次被使用的时间间隔较长，且如果网络模型的层数越多，该种数据空闲的时间间隔越长，因此，可以使用重计算技术对特征数据进行内存优化。

以优化 T2 数据为例，T2 是前向传播中的计算层 F2 的输出数据，当 T2 数据输入给计算层 F3 之后，会出现长时间未被使用，但是仍然保存在 GPU 内存中的情况，重计算技术会保留计算层 F2 的输入数据，并在 T2 数据输入给计算层 F3 之后将该数据直接丢弃，在反向传播计算层 B2 和 B3 需要使用 T2 数据之前，通过重新执行计算层 F2 的方式生成特征数据 T2，并将该数据做为 B2 和 B3 层的输入，通过重计算技术对数据进行内存优化，能有效减少内存瓶颈时期的内存使用量，避免模型训练过程出现内存不足而中断训练的情况。

2.2.2 内存交换技术

内存交换（Memory Swapping）技术是深度学习领域中用于优化内存使用的一种关键方法，特别适用于在内存受限的环境中训练大型神经网络模型。该技术的基础思想是在 GPU 和 CPU 之间动态地交换数据，以此来减轻 GPU 内存的负担，从而使得更大或更复杂的模型能够在有限的硬件资源上进行训练。

在实际的训练过程中，数据会从 CPU 中加载到训练设备如 GPU 中，当计

算完成之后，会从 GPU 设备的内存中拷贝回 CPU 中，由于模型训练过程中的数据通常都会被频繁使用，所以会将训练过程中产生的中间数据都保留在 GPU 设备的内存中，而内存交换技术则在训练开始之前，算法会确定哪些数据应该存储在 GPU 内存中，哪些可以暂时存储在 CPU 内存中。这一决策通常基于数据的重要性和访问频率，以及 GPU 和 CPU 之间的数据传输成本。在模型的前向传播阶段，只有将必要的数据保留在 GPU 内存中，其他数据则通过 PCIe 总线交换至 CPU 内存中。随着模型进入反向传播阶段，那些之前被交换到 CPU 内存的数据将根据需要被重新加载回 GPU 内存。

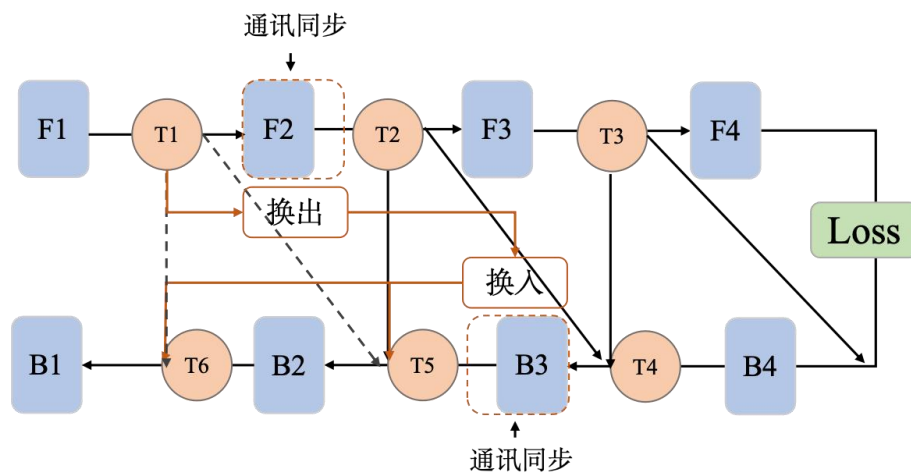


图 2.2 内存交换技术示意图

如图 2.2 中所示，选择使用内存交换技术优化前向传播中的计算层 F1 的输出数据 T1，那么在计算层 F2 接收到 T1 作为输入数据之后，T1 会被换出到 CPU 内存中，当换出的数据传输流程结束，当前 GPU 训练设备中 T1 所占用的内存空间会被释放。随后在反向传播过程中，计算层 B2 和 B1 需要使用到特征数据 T1 之前，GPU 设备会重新为 T1 分配内存空间，并将数据从 CPU 中换入回 GPU 内存，进行正常的反向传播流程。

2.3 自适应内存优化方法

为了进一步提高内存优化算法在不同的模型结构和数据访问模式中的优化效率，学者提出了自适应内存优化方法来智能地组合和分配内存优化策略，从而实现高效的资源管理策略。本文将围绕当前业界先进的 Capuchin 方法，POET 方法，HOME 方法以及深度学习训练框架 TensorFlow 的自适应内存优化方法展开介绍。

(1) TensorFlow 的自适应内存优化方法

TensorFlow 是一个由 Google Brain 团队开发，被广泛应用于深度学习模型

开发与部署的 AI 计算框架。由于观察到模型训练过程中内存资源紧张的问题，TensorFlow 开发团队在框架中实现了基于重计算和内存交换思想的内存优化方法，TensorFlow 中提供了 `memory_optimization` 配置用于决定如何对模型的训练过程进行内存优化，该方法可以传入参数列表优化指定的数据，也可以通过自适应的方式对模型图进行优化，下文将介绍 TensorFlow 中的自适应内存优化方式。

在 TensorFlow 中，用户定义的模型结构由名为图（Graph）的数据结构进行表示。图（Graph）是整个 TensorFlow 计算任务的基础，它是一个由节点（Nodes）和边（Edges）构成的网络结构。节点在图中代表不同的数学操作，如加法或乘法，而边则表示节点间的数据关系，即数据以多维数组的形式（张量，Tensor）在节点间流动。张量作为 TensorFlow 的核心数据结构，实际上是一个可以有任意维度的数组，负责携带数据流经图中的各个节点。TensorFlow 中的内存优化方法的输入就是一个由图表示的模型计算图，该方法根据图结构和设备信息按顺序进行内存优化。首先进行重计算优化，随后针对累加和累乘计算节点的内存使用情况进行调整，在仍不满足训练设备内存限制的情况下，将采用内存交换技术来进一步优化模型计算图的内存占用。

首先，在 TensorFlow 的内存优化方法中，指定了一个静态列表，列表中包含了一系列计算代价较小的计算节点类型，如 Add、Sub、Mul、Div、Reshape、Rule、Sigmoid 等等。在进行重计算优化时，当模型计算图中包含了对应类型的计算节点，并且这些计算节点的输出数据将在反向传播过程中重用时，TensorFlow 会将这些计算节点的输出作为优化目标，通过重新计算对应节点的方式减少内存占用。

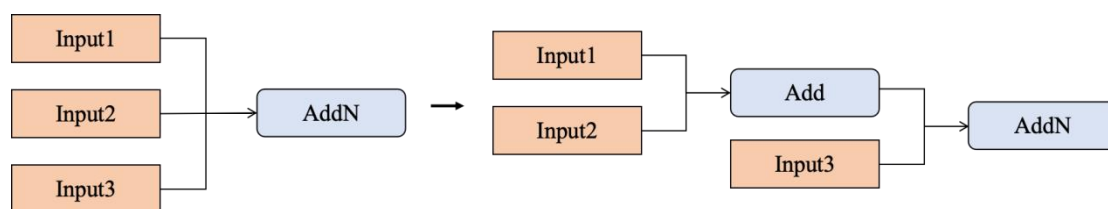


图 2.3 累加节点内存优化示意图

其次，TensorFlow 中对累加和累乘计算节点做了额外的内存优化。这类计算节点的特点是有多个输入数据，这意味着在执行累加或者累乘计算时，GPU 设备中需要有足够的内存空间来存储所有输入数据。为避免因内存不足导致训练中断，TensorFlow 在保证数据准确性的前提下，将多个输入数据的单一计算操作转化为两个输入数据的多个线性计算操作。例如，对于一个包含三个输入数据 {Input1, Input2, Input3} 的累加计算节点，其输出值为三个输入之和，TensorFlow 将引入一个新节点来计算 Input1 和 Input2 的和，新增的 Add 节点的输出值将与 Input3 一起作为原累加节点的输入，通过这种方式，该累加计算

过程中的输入数据的内存占用降低到优化前的 $2/3$ 。

最后，当通过重计算技术和累加累乘优化技术对模型计算图的优化仍无法满足内存需求时，TensorFlow 将采用内存交换技术，进一步优化模型图的内存使用情况。具体来说，算法首先遍历那些在训练过程中被多次使用的特征数据。通过分析最后一个前向传播生成的特征数据的时间点，算法估计内存使用的峰值时间 $peak_time$ 。然后在排除那些体积小且快速释放的特征数据后，按照式 (2.4) 计算特征数据用于内存交换的合适度 $fitness$ ，并根据 $fitness$ 倒序选择数据应用内存优化策略，直至优化后的模型训练内存峰值低于设备内存容量。

$$fitness = (earliest_use - peak_time)^2 / use_left^2 + (allocation_time - peak_time)^2 \quad (2.4)$$

式 (2.4) 中， $earliest_use$ 表示数据最早使用时间， $uses_left$ 表示数据剩余使用次数， $allocation_time$ 表示数据实际分配内存空间的时间。不难发现，该公式筛选离内存使用峰值尽量远的数据，也就是说，TensorFlow 中更倾向于优化在前向传播中较早生成的特征数据。

(2) Capuchin

Capuchin 是一个基于 TensorFlow 实现的自适应内存优化框架。Capuchin 方法动态跟踪张量访问信息，并运用贪心算法为各张量分配优化策略，以减少内存占用。该方法将模型训练过程划分为两个阶段：测量执行阶段和指导执行阶段。在测量执行阶段，Capuchin 在小 batch size 训练过程中收集动态张量访问序列，并基于此制定内存管理策略。随后，在指导执行阶段，这些策略被用于引导实际的训练过程。Capuchin 方法为不同结构的模型提供针对性的内存优化方案，实现了精细和灵活的内存管理。

在测量执行阶段，Capuchin 方法收集张量的访问信息，包含张量的访问频次、时间戳和相关操作信息等等，这些信息被用于估算模型训练过程中的内存使用峰值，内存使用峰值减去 GPU 设备内存容量即为内存优化目标。为实现此目标，Capuchin 采用重计算和内存交换这两种方法对张量内存占用进行优化。优化过程中，首先为所有重用张量寻找合适的数据换入和数据换出的时机，针对没法找到换入换出时机的张量，再比较由式 (2.5) 计算的内存交换方法优化收益和式 (2.6) 得出的重计算方法优化收益，选择较优策略作为该张量的内存优化方法。

在 Capuchin 方法中，寻找数据换入和数据换出的触发时机是关键步骤。在寻找数据换出时机时，首先通过张量尺寸除以设备总线通信带宽计算出张量数据的交换时间 $swapTime$ ，寻找晚于张量数据正向最后一次访问的张量数据作

为数据换出的触发时机，该触发时机加上 $swapTime$ 就是 $SwapOutEndTime$ ，这代表张量数据驱逐完成的时间。在寻找数据换入时机时，则由后向传播过程中的访问时间减去张量所需的 $swapTime$ 来确定。这意味着 Capuchin 会在张量实际需要之前的某个时间点重新生成，以确保在需要时张量已经可用。该时间点需早于后向传播访问时间减去张量数据交换时间 $swapTime$ ，Capuchin 会从后向前遍历张量访问列表，寻找第一个满足条件的张量，使用该张量的时间戳作为 $SwapInStartTime$ 。可以观察到式 (2.5) 中计算的 FT 实际上代表了数据换出到 CPU 中的时间段长度。

$$FT = SwapInStartTime - SwapOutEndTime \quad (2.5)$$

$$MSPS = MemorySaving / RecomputationTime \quad (2.6)$$

在无法找到满足上述条件的数据换入和换出时机时，则需要比较张量使用重计算技术和内存交换技术的收益。Capuchin 在使用重计算技术进行内存优化的过程中不设置触发时间，只在必要时进行，以减少对计算资源的额外需求。对于重计算时间的估算需要考虑张量间的依赖关系，使得这个过程相对复杂。例如，在张量 $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$ 的线性计算依赖中，重计算 $T4$ 的成本取决于所选择的重计算源。具体来说，如果 $T4$ 的重计算源是 $T3$ ，那么 $T4$ 的重计算时间 $RecomputationTime$ 就是 $T3$ 到 $T4$ 的时间间隔，但是如果 $T3$ 因为重计算或者内存交换方法移出了 GPU 内存，那么 $T4$ 的重计算时间 $RecomputationTime$ 则为 $T2$ 到 $T4$ 的时间间隔。正是这种数据依赖性使得重计算时间的估算变得更加复杂。式 (2.3) 中 $MemorySaving$ 代表了重计算数据节省的内存大小，该式表示了对张量执行重计算方法每秒节省的内存。

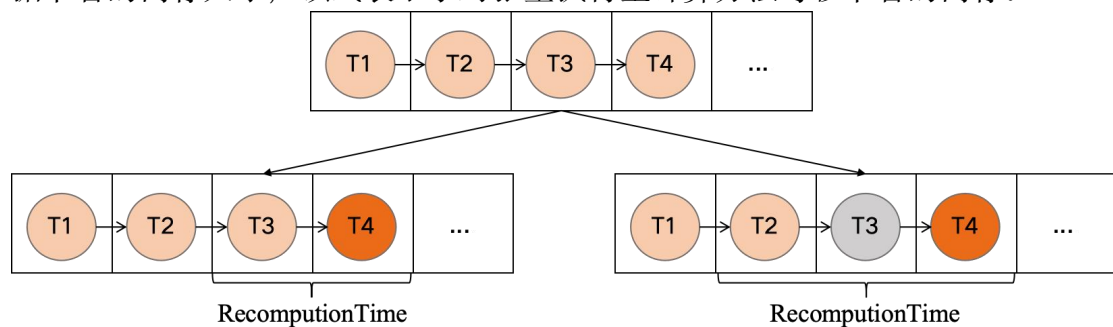


图 2.4 $RecomputationTime$ 计算示意图

在指导执行阶段，测量执行阶段生成的优化策略被用来指导如何更有效地管理内存，例如何时交换（驱逐）和预取（重新加载/生成）张量。Capuchin 对 TensorFlow 的内存分配器（Allocator）进行了拓展，使其支持更复杂的内存管理操作，如张量的驱逐和预取，并在执行器（Executor）的张量计算过程前后插入额外的逻辑，以跟踪和管理张量的访问和重计算过程。

(3) POET

POET 算法是一个针对边缘设备训练场景的自适应内存优化方法，该方法集成重计算和内存交换两种技术来减少反向传播过程中的内存消耗，利用混合整数线性规划（MILP）生成内存优化策略，从而实现在内存受限的边缘设备上训练大型神经网络。POET 方法具有创新性地使用了矩阵的方式来表示模型计算图 $G(V, E)$ 中的拓扑结构和数据依赖关系，在设置的训练设备内存预算 μ_{RAM} 和时间预算 $\mu_{deadline}$ ，以及保证反向传播的数学正确性的前提下，找到一个能够在内存受限的设备上训练出较大模型的方案。

首先，POET 方法中基于模型计算图的结构 $G(V, E)$ 创建了 MILP 算法搜索基础约束条件，以保证模型的正常训练过程。 $G(V, E)$ 中， V 代表模型计算图中的计算节点集合，而 E 则是指有数据依赖关系的节点的边集合。假设有计算图中计算节点数量总共有 T 个，那么使用 5 个矩阵 $\{0,1\}^{T \times T}$ 分别表示模型的计算状态 R ，训练设备主存使用情况 S_{RAM} ，辅助存储器使用情况 S_{AUX} ，设备数据换出状态 M_{in} ，数据换入状态 M_{out} 。以计算状态 R 为例， $R_{t,i} = 1$ 代表在节点 V_i 在 t 时刻执行了计算或者重计算操作，类似的， $M_{t,i}^{out} = 1$ 表示在时间 $t-1$ 和 t 之间将张量从到主存 RAM 卸载到 AUX。 $S_{t,i}$ 则表示了在 t 时刻节点 V_i 是否保存在对应存储器中。固定 $S_{1,i} = 0$ 表示训练开始阶段内存中未保存任何特征数据，设置 $R_{v,v} = 1 (\forall v \in V)$ 来代表模型的正常训练过程中的计算状态。

$$R_{t,i} + S_{t,i}^{RAM} \geq R_{t,j} \quad \forall t \in V \quad \forall (v_i, v_j) \in E \quad (2.7)$$

$$R_{t-1,i} + S_{t-1,i}^{RAM} + M_{t-1,i}^{in} \geq S_{t,i}^{RAM} \quad \forall k \in K \quad \forall t \geq 2 \quad \forall i \quad (2.8)$$

$$S_{t-1,i}^{AUX} + M_{t-1,i}^{out} \geq S_{t,i}^{AUX} \quad \forall t \geq 2 \quad \forall i \quad (2.9)$$

$$S_{t,i}^{AUX} \geq M_{t,i}^{in} \quad \forall k \in K \quad \forall t \geq 2 \quad \forall i \quad (2.10)$$

$$S_{t,i}^{RAM} \geq M_{t,i}^{out} \quad \forall k \in K \quad \forall t \geq 2 \quad \forall i \quad (2.11)$$

然后，在上述的模型训练状态矩阵的基础上，POET 方法提出了一系列条件以保证使用重技术技术和内存交换技术优化后的模型能够正常训练：如式 (2.7) 所述，如果节点 (V_i, V_j) 间存在数据依赖关系，那么节点 V_j 在 t 时刻执行计算操作时，对应的依赖节点 V_i 应当已经存在于 RAM 内存中或者正在通过计算生成。并且针对重计算和内存交换优化技术，提出以下约束（以下约束条件对应于式 (2.8) 至 (2.11)）：

(1) 对于 t 时刻位于 RAM 内存中的节点 V_i 的数据，该数据只能由三种方式生成： $t-1$ 时刻计算生成； $t-1$ 时刻从 AUX 内存中换入生成；或者 $t-1$ 时刻数据已经存在 RAM 内存中。

(2) 对于 t 时刻位于 AUX 内存中的节点 V_i 的数据, 该数据只能是在 $t-1$ 时刻从 RAM 内存中换出生成; 或者 $t-1$ 时刻数据已经存在 AUX 内存中。

(3) 在 t 时刻, 节点 V_i 的数据只有存储在 AUX 中才可以执行换入操作。

(4) 在 t 时刻, 节点 V_i 的数据只有存储在 RAM 中才可以执行换出操作。

最后, 式 (2.12) 和式 (2.13) 限制设备最大训练内存和训练执行时间需小于内存预算 μ_{RAM} 和时间预算 $\mu_{deadline}$ 。根据以上约束条件, MILP 算法会找出一组满足约束条件的解, 其中矩阵 M_{in} 和 M_{out} 中标识了内存交换的优化策略, 矩阵 R 则包含了正常计算和重计算的相关信息, 通过策略搜索结果对模型的计算图结构进行优化, 并在边缘设备上训练优化后的模型以减少资源消耗。

$$U_{t,i}^{RAM} \leq \mu_{RAM} \quad \forall t \in V \quad \forall i \in V \quad (2.12)$$

$$\sum_T [R\psi_{compute}]_T \leq \mu_{deadline} \quad (2.13)$$

(4) HOME

HOME 方法是一个为深度学习训练场景设计的自适应内存优化技术。该方法结合张量交换和重计算技术, 依据模型结构和运行信息, 实现高效内存优化。HOME 方法采用粒子群算法优化内存优化技术的放置策略, 旨在不牺牲计算效率的前提下, 增加模型训练的批处理规模。

首先, HOME 方法在第一次迭代中搜集了模型结构和运行时的详细信息, 包括但不限于特征图的大小、每个核心的执行时间、特征图之间的依赖关系以及 GPU 与 CPU 之间的数据传输带宽等等。在获取这些关键信息之后, 该方法采用粒子群优化 (PSO) 算法^[49]来确定张量放置策略。为了有效应用 PSO 算法, HOME 方法将特征数据内存优化问题转化为一个数学上的分配问题, 使用 0、1 和 2 三个数字来表示不同的张量数据的放置行动, 即保留、内存交换和重计算。例如, 张量放置策略 [1, 1, 2, 0] 表示在前两个特征图上进行交换, 在第三个特征图上进行重计算, 在第四个特征图上进行保留。这样的向量可以被视为在高维搜索空间中的一个粒子, 其维数等于总特征图的数量。因此, 寻找张量放置策略可以被视为寻找粒子的位置。在初始化时, 生成多组随机粒子序列用于算法的演化搜索。

然后, HOME 方法借助双向演化的 PSO 算法调整张量放置策略, 分别根据全局最优及历史最优来指导演化搜索过程。具体来说, 在每次迭代中, HOME 方法利用 PSO 算法的更新过程, 首先识别并记录下当前迭代中所有粒子之间的最优放置策略 (全局最优), 以及每个粒子在过去迭代中的最优放置策略 (历史最优)。如图 2.5 所示, 对于张量放置序列中的第一个张量, 也是第一个粒子, 考虑它在当轮 (第二轮) 搜索过程中最优张量放置序列中的放置选择, 也

考虑该粒子所在张量放置序列历史演变过程中的最优序列中的放置选择，算法分别比较当前张量的放置行为与这两个极值策略中对应的历史行为，评估替换后的训练开销，并选择导致较小开销的历史行为来更新当前张量的放置行为。通过这种方式，根据更小训练开销的历史策略更新每个粒子的张量放置行为，形成新的放置策略。其中，对于训练开销的评估方法主要遵循以下规则：对于内存交换技术带来的训练开销，在正常训练时间上新增无法与计算时间重叠的通信时间；对于重计算技术带来的训练开销，则在正常训练时间上累加重计算时间，该时间与前文 Capuchin 方法中 *RecomputationTime* 的计算方法相同。

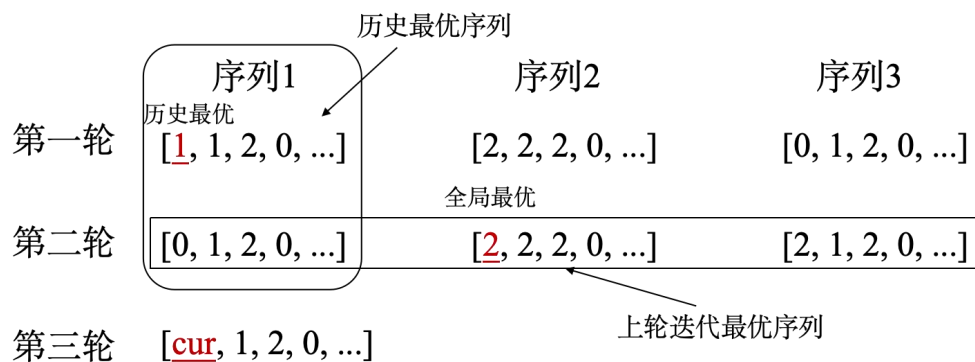


图 2.5 PSO 算法双向演化过程示意图

最后，重复执行该过程，根据双向演化的 PSO 算法更新每个粒子的张量放置行为，直到所有粒子的放置策略均找到最优解，并在实际训练环境中选择该张量放置策略进行应用。

2.4 内存分配算法

深度学习模型的训练过程呈现重复迭代的性质，导致数据的频繁分配与释放成为常态。训练设备系统级的内存管理操作，例如 `cudaMalloc` 和 `cudaFree`，通常具有较高的执行成本，同时对内存的利用率较低。鉴于此，研究人员提出优化的内存分配算法，通过高效管理内存空间，降低了训练过程中的性能开销。

(1) TensorFlow 的内存分配算法

TensorFlow 中的 `BFCAllocator` (Best Fit with Coalescing Allocator) 是一种基于分块思想的内存分配算法，旨在高效地分配和管理 GPU 或 CPU 内存资源。它通过逻辑和物理两个方式管理内存块的使用情况，选择最适合请求大小的空闲内存块进行分配，并通过合并 (Coalescing) 相邻的空闲块来优化内存的使用。从而减少内存碎片，确保内存使用的高效性，提高深度学习模型训练和推理的性能。

在 `BFCAllocator` 中，同时包含 `Chunk` 和 `Bin` 两种结构来共同管理内存。

其中, Chunk 是 BFCAllocator 管理的基本内存单元, 代表了一块连续的内存区域。每个 Chunk 包含了内存的起始地址、大小、以及一些状态信息(如是否被分配)。Chunks 在物理上通过前后指针相互链接, 形成链表结构。Bins 则是用来组织不同大小范围内内存块的逻辑容器。Bins 通常按照指数规模划分, 在 TensorFlow 中, 总共设置了 21 个 Bin, 依次从 256B 按 2 的倍数增加到 512M。BFCAllocator 根据 Chunk 的大小将其分类到不同的 Bins 中, 这样可以快速定位到合适大小的 Chunk, 提高内存分配的效率。

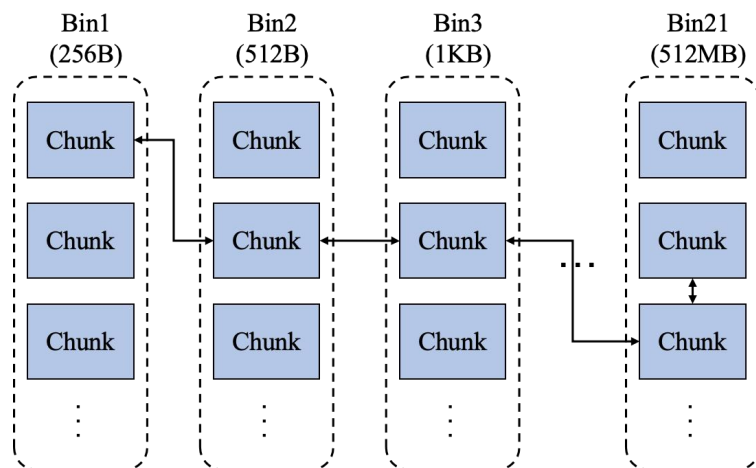


图 2.6 BFCAllocator 数据结构示意图

在分配请求过程中, BFCAllocator 首先按照 256B 的大小对齐内存请求, 然后根据请求的内存大小找到合适的 Bin, 然后从该 Bin 中选择一个合适的 Chunk 进行分配。如果找到的 Chunk 符合切分条件, 即 Chunk 的大小是请求大小的两倍或者 Chunk 的大小比请求内存大 128M, 则 BFCAllocator 会将该内存块分割为两部分: 一部分正好满足请求大小, 另一部分会作为一个新的 Chunk 加入到合适的 Bin 中。

在释放请求的过程中, BFCAllocator 会将相应的 Chunk 标记为未分配, 并尝试与物理上相邻的空闲 Chunk 进行合并, 合并后的大 Chunk 会被重新分类到合适的 Bin 中。这种管理方式旨在动态调整内存布局, 尽可能减少内存碎片, 提高内存使用效率。

(2) PyTorch 的内存分配算法

PyTorch 的 CudaCachingAllocator 是一种专为 GPU 设备设计的内存分配器, 该方法通过维护一个内存池, 实现了对 GPU 内存的高效管理。它缓存了被释放的内存块, 并在后续的内存请求中复用这些内存块, 以减少对 cudaMalloc 和 cudaFree 调用的依赖。这种策略显著降低了内存碎片, 并提高了内存分配的速度。

Block 是 CudaCachingAllocator 管理的基本内存单位, 代表了一块连续的

GPU 内存区域。每个 Block 都有特定的大小和起始地址，并且可以被标记为使用中或空闲，每个 Block 可以根据需求切分成多个 Block，并且通过前后指针相互标记。CudaCachingAllocator 中维护两个内存池用于管理 Blocks，小型内存池负责管理小于 2MB 的 Blocks，大型内存池中负责管理大于 2MB 的 Blocks，通过这样的组织方法可以快速定位到合适大小的内存池，以满足特定的内存请求。

在内存分配请求的处理过程中，CudaCachingAllocator 首先将内存请求按照 512 字节的大小进行对齐，随后定位至相应的内存池。其次，该分配器在指定的内存池中查找与请求尺寸最接近的空闲块以进行分配。具体来说，如果找到的空闲块位于小型内存池中且其大小超过请求的 512 字节，则该块会被分割为两部分：一部分用于满足当前请求，而剩余部分则继续保留在内存池中以供后续使用。相对地，若空闲块位于大型内存池中且其大小超出请求的 1MB，则会将此块切分。在找不到足以满足请求的空闲块时，CudaCachingAllocator 会调用 cudaMalloc 向 GPU 设备请求新的内存空间。若设备也无法满足该分配请求，分配器则会尝试将未切分的块返回给设备后再次申请分配。

在处理释放请求时，相应的内存块不会立即返回给操作系统，而是重新归入 CudaCachingAllocator 的相应内存池，并标记为可用状态。此外，该内存块会与其相邻的空闲块进行合并，从而优化未来的内存请求处理，实现资源的有效复用。

(3) PagedAttention

PagedAttention^[30]方法主要针对大语言模型训练过程中的序列的键值缓存进行内存优化。以操作系统中的虚拟内存和分页的经典思想为灵感，对模型训练数据进行管理。

在大语言模型进行训练的过程中，通常要存储模型训练的中间数据，中间数据包括与注意力机制相关联的键和值张量，通常称为 KV 缓存，KV 缓存具有独特的特征：随着模型生成新令牌，它会动态增长和收缩，其生命周期和长度并不是事先已知的。这些特征使得现有系统受到内部和外部内存碎片化的困扰。灵感来自于操作系统中虚拟内存和分页的经典思想，它可以允许在非连续空间立存储连续的 KV 张量。具体来说，PagedAttention 把每个序列的 KV 缓存进行了分块，每个块包含固定长度的令牌，而在执行注意力层的计算时可以高效地找到并获取那些块。因此，PagedAttention 可以像操作系统的虚拟内存一样更灵活地管理 KV 缓存：可以将块视为页面、令牌视为字节、请求视为进程。这种设计通过使用相对较小的块并根据需要分配它们来减轻了内部碎片化，并完全消除了外部碎片的问题。

PagedAttention 对请求空间预先分块，并使用块表对内存空间进行管理。在内存分配请求的处理过程中，根据块内尺寸对请求的数据进行切分，并将切分后的数据分散到不同的空闲内存块中去，将请求和物理内存块的对应关系进行记录。并且 PagedAttention 方法对于大语言模型中的注意力层进行改造，添加了并行读取分块存储中的数据的逻辑，以加速分块存储情况下的数据读取过程。在处理释放请求时，则通过块表快速定位数据实际存储的内存块，标记物理空间为空闲状态，方便后续的分配请求重新使用该空间，PagedAttention 使用块级内存管理方法，在存储大语言模型训练过程中的 KV 缓存是实现几乎零浪费的内存存储。

2.5 强化学习算法

强化学习^[56] (Reinforcement Learning, RL) 是常用的策略搜索算法。它主要研究如何让智能体 (agent) 通过与环境的交互来学习最优策略，从而实现目标的最大化累积回报。与传统的监督学习和非监督学习不同，强化学习不依赖于预先标记的数据集，而是通过智能体在环境中采取行动并接收回馈 (奖励或惩罚) 来学习。

基于值函数的强化学习算法是强化学习中的一大类方法，其核心思想是通过评估在给定状态下采取不同行动的价值，来指导智能体做出决策。这里的“值”一般指的是期望回报，即从当前状态出发，采取某个行动并遵循某个策略所能获得的累积奖励的期望值。经典的基于值函数的强化算法包括 Q 学习^[57] (Q-learning) 和时序差分学习^[58] (TD Learning)，深度 Q 网络^[59] (DQN) 等等。这类算法的关键在于如何准确估计状态-行动对 (state-action pair) 的价值函数，从而引导智能体学习最优策略。

其中，DQN (Deep Q-Network) 算法是一种结合深度学习和增强学习的方法，用于解决序列决策问题。其核心思想基于 Q-learning，通过一个深度神经网络来近似 Q 函数。Q 函数，记作 $Q(s, a)$ ，表示在状态 s 下采取动作 a 所能获得的预期回报。DQN 的目标是找到一个策略序列，最大化累积回报。DQN 算法的关键公式为 Q 值的更新规则，其通过以下方式进行迭代更新：

$$Q_{new}(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma * \max_a Q(s_{t+1}, a) - Q(s_t, a_t)) \quad (2.14)$$

其中， s_t 和 a_t 分别代表在时间 t 的状态和采取的动作， r_{t+1} 是执行动作后获得的即时奖励， γ 是未来奖励的折扣因子， α 是学习率，用于平衡旧的 Q 值和

新信息的重要性。同时，DQN 算法引入了两个关键的技术以解决深度学习中的稳定性和收敛问题：经验回放^[60]（Experience Replay）和固定 Q 目标（Fixed Q-targets）。经验回放通过随机采样过去的转换经验来打破样本之间的相关性，而固定 Q 目标通过维护一个定期更新的目标网络来减少目标值的波动性，从而学习出有效的策略序列。

面对复杂的问题搜索环境，学者提出多智能体强化学习^[61]（Multi-Agent Reinforcement Learning, MARL）方法。这类方法关注的是多个智能体在同一个环境中相互作用时的学习问题。在多智能体系统中，每个智能体的决策不仅取决于环境，还取决于其他智能体的行为。这增加了问题的复杂度，因为每个智能体的策略更新都可能影响其他智能体的最优策略。多智能体强化学习的目标是学习一组策略，这组策略能够在多智能体交互的环境中实现某种形式的最优或平衡。针对 MARL 问题，研究者们提出了多种算法，如独立 Q 学习^[62]（Independent Q-Learning）、多智能体深度确定性策略梯度^[63]（Multi-Agent Deep Deterministic Policy Gradient, MADDPG）等等。

2.6 本章小结

本章首先介绍了深度学习模型训练方法，其次介绍了内存优化技术的相关基础，包括两种轻量化的内存优化技术：重计算技术和内存交换技术。然后介绍了多种自适应内存优化算法。随后本文介绍了内存分配算法的相关基础。最后介绍本文研究方法中所使用的强化学习算法。

第三章 基于双智能体强化学习的自适应内存优化方法

3.1 问题描述

随着深度学习在生产生活中广泛应用，为了提高模型预测的准确度，深度学习模型的规模成指数性增长，然而训练设备内存资源的增长速度则相对缓慢，模型训练时内存需求与设备内存资源之间存在极大的不平衡，如何利用有限内存资源实现更加高效地模型训练是一个重要命题。为了应对这一挑战，研究人员提出多种内存优化技术，以及结合这些技术的自适应内存优化方法对模型训练过程中产生的数据进行管理。然而，现有方法在应对模型结构差异方面仍有所不足。为此本文针对模型训练场景下，现有内存优化方法与模型结构之间的关系进行分析如下：

(1) 模型训练过程中不同数据所呈现的优化潜力具有差异性。以 VGG16^[64]模型为例，图 3.1 中按照层执行顺序列出了模型中的不同层在训练过程中的输出数据尺寸。具体而言，模型中较靠近输入层的部分往往需要处理更大尺寸的特征数据，这些数据直接从原始输入中提取，内存占用较高，且经过一段时间后才会被重复使用，能够保持较长的释放时间，优化价值较高。随着模型的正向传播训练过程，特征表示逐渐抽象化，每一层加工后的特征数据虽然在语义层面更加丰富，但数据量相对减少，并且，接近输出层的数据会在反向传播过程中被快速重用，需要在数据释放优化后短时间内重新生成，优化价值相对较低。

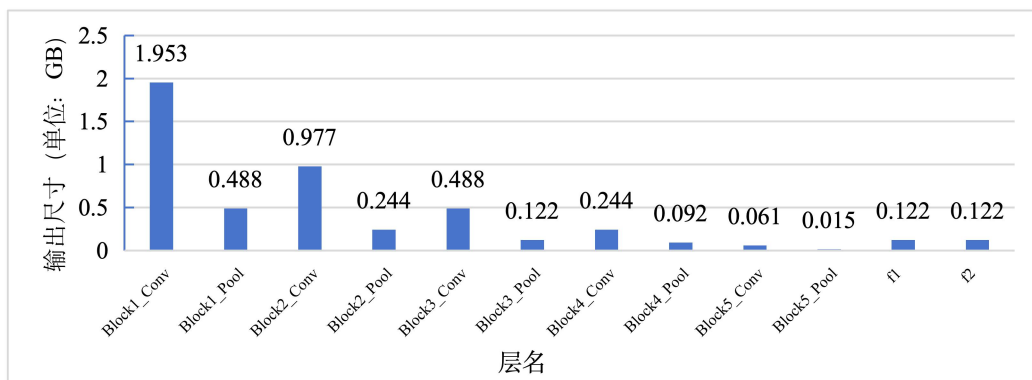


图 3.1 VGG16 模型中各层输出尺寸

(2) 目标优化数据应用不同内存优化技术的优化代价呈现多样性。重计算和内存交换技术在在扩大可用内存容量的同时需要引入额外的计算代价和通信代价。以计算节点 F_i 为例，在执行重计算内存优化时，会在正向传播过程中释

放 F_i 的输出数据，并在反向使用到该输出数据之前，根据 F_i 的输入数据重新计算生成。当 F_i 的输入数据均驻留在内存中时， F_i 的重计算代价即为该计算节点的执行时间。如式 (3.1) 中所示，该数据由 F_i 计算所需的浮点运算次数和设备浮点运算能力决定，因此， F_i 的重计算代价与该节点的计算类型相关。对计算节点 F_i 使用内存交换技术进行优化时，所需的代价为 F_i 输出数据进行数据换入和数据换出时的传输成本。如式 (3.2) 中所示，内存交换代价由 F_i 的输出数据尺寸，CPU 和 GPU 的传输带宽决定，这意味着， F_i 的内存交换代价同该节点的输出尺寸相关。综上，为计算节点应用内存优化技术需要根据实际情况进行考虑，这进一步增加了资源配置的挑战。

$$\begin{aligned} \text{重计算代价: } Cost_{F_i} &= \text{execute time}_{F_i} \\ &= FLOPS_{F_i} / \max FLOPS * UtilRate \end{aligned} \quad (3.1)$$

$$\text{内存交换代价: } Cost_{F_i} = \text{Output size}_{F_i} / \text{bandwidth} \quad (3.2)$$

(3) 执行内存优化的时机对于内存优化效率具有显著影响。在使用内存交换技术进行内存优化的过程中，通常通过多个 CUDA 流来重叠数据通信过程与正常计算过程，然而为了防止数据拷贝过程中出现错误，对于无法同计算重叠的通信进程，则需要额外的同步时间来等待数据传输进程完成。并且，当使用重计算技术进行优化过程时存在依赖数据也被释放的情况，比如节点 F_{i-1} 的输出数据也进行了内存优化，那么 F_i 的重计算过程则需从 F_{i-2} 开始，这会延长重计算链条从而增加训练时间。

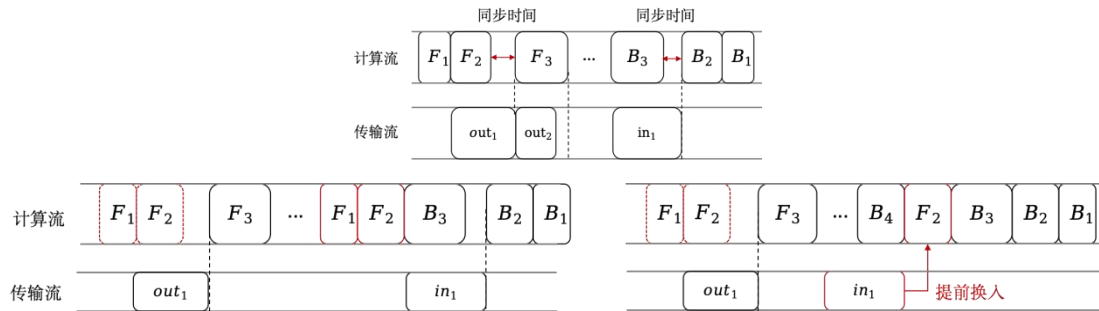


图 3.2 内存优化执行时机对训练时间的影响

根据上文所分析的模型结构与内存优化方法的特征关系，本文总结了现有深度学习模型训练场景下的自适应内存优化方法需要解决的三个问题：

(1) 如何选择合适的数据进行内存优化，从而最大化数据优化效率？模型训练过程中数据所呈现的优化潜力差异性要求内存优化方法能够灵活调整，以便在不同的模型训练阶段中选择合适的数据进行优化，在实现在保证优化效果的同时，最小化所需付出的代价。

(2) 对优化数据选择何种内存优化技术能减少额外代价？有效的自适应内

存优化方法必须考虑到模型不同层级上特征数据处理的具体需求，以及这些需求如何随模型结构的深度和复杂度而变化。通过精细化管理模型的优化技术分配方法，可以显著提升训练过程的效率和速度，同时减少资源浪费。

(3) 如何选择内存优化策略执行的时机以优化训练效率？理想的执行时机能够保证在减少内存需求同时不影响训练时间。比如选择能够与计算进行重叠的时机进行内存交换能够减少数据通信同步时间，这要求对不同内存优化技术的适时应用，以实现更为经济高效的模型训练过程。

围绕模型训练场景下的上述问题，本文提出的基于双智能体强化学习的自适应内存优化方法 REMEM(Reinforcement learning based GPU Memory Management Algorithm)。

3.2 REMEM 方法概述

针对上述问题，本研究提出基于双智能体强化学习的自适应内存优化方法，REMEM(Reinforcement learning based GPU Memory Management Algorithm)。该方法综合利用重计算与内存交换技术，并结合模型结构与设备特性信息，通过强化学习算法探索并确定最佳内存优化策略序列及其执行时机。方法的具体流程如下：

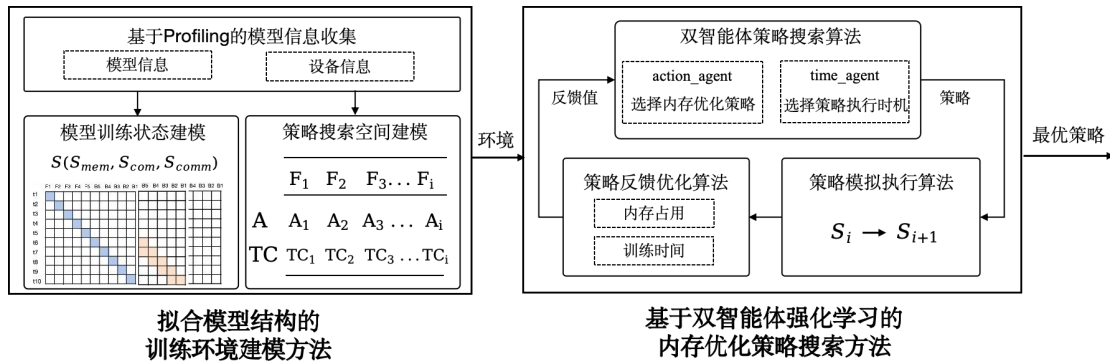


图 3.3 基于双智能体强化学习的自适应内存优化方法流程

首先，基于 Profiling 技术收集模型训练过程中的数据信息。根据数据信息，模型训练状态建模方法初始化多维状态矩阵 $S(S_{mem}, S_{com}, S_{comm})$ 用于模拟真实环境中的模型训练过程；内存优化策略搜索空间建模方法则根据收集的数据信息确定特征映射数据的优化策略搜索空间 A 和策略执行时机搜索空间 TC 。模型训练状态和策略搜索空间共同组成强化学习算法的搜索训练环境。

其次，在构建的搜索环境中，本研究采用了一种双智能体策略搜索算法，该算法设计使用 $action_agent$ 为特征映射数据选择内存优化策略，使用 $time_agents$ 确定策略执行时机。基于选择的策略，进一步引入了策略模拟执行

算法，通过更新多维状态矩阵 $\{S_i \rightarrow S_{i+1}\}$ ，以模拟内存优化方法在实际环境中的执行效果。策略反馈优化算法通过模拟执行结果计算反馈值，以此为依据对策略搜索结果进行迭代优化，最终确定并实施最优的内存优化策略。

基于强化学习的自适应内存优化方法如图 3.3 所示。在 3.3 节和 3.4 节中，本文将进一步阐述拟合模型结构的训练环境建模方法和基于双智能体强化学习的内存优化策略搜索方法。

3.3 拟合模型结构的自适应内存优化问题建模方法

REMEM 采用了强化学习方法自适应地探索模型训练过程中的最优内存优化策略。而强化学习框架高度依赖于训练环境进行策略的学习、搜索和性能反馈。为了解决内存优化问题，本研究收集模型训练数据，拟合模型结构的对自适应内存优化问题进行建模。

3.3.1 基于 Profiling 的模型信息收集

对于深度学习来说，“学习”指的是通过调整神经网络模型中的权重参数，以最小化模型预测输出和样本数据间的差异。在深度学习过程中，需要迭代遍历所有的样本数据，样本数据被随机分为多个“batch”，而“batch size”是指每次迭代训练模型时所使用的输入样本数据的数量。在训练过程中，正向传播过程特征数据和反向传播过程中梯度数据的数据尺寸都与 batch size 线性相关，所以 batch size 是验证内存优化效果的一个重要指标。

在内存资源有限的设备上扩大模型训练的 batch size，需要使用重计算和内存优化这样的技术对于模型的训练过程的数据存储情况进行优化，而数据在模型中的位置，数据的计算代价，数据尺寸，均会影响内存优化技术的执行效率。因此，本文基于 Profiling 技术^[65]收集整理模型信息，以辅助内存优化的策略决策过程。

表 3.1 模型信息概述

信息名称	描述
执行时间 (execute time)	计算节点运行所需时间，使用线性回归模型拟合
输出尺寸 (output size)	计算节点所有输出张量占用的内存空间尺寸
数据依赖 (data dependency)	计算节点间的数据依赖关系

鉴于需要训练尽量大 batch size 的数据来对策略效果进行验证，而未进行优化的模型图无法完成该任务，本文收集了多个不同 batch size 下的模型运行

数据，以拟合大 batch size 下的模型训练信息。神经网络模型会在真实环境中多次执行不同 batch size 的训练过程，通过 Profiling 技术采集模型执行日志，Profiling 技术在计算加速设备执行计算节点的前后进行数据收集，记录各个计算节点的耗时，输入输出等信息。通过对多个 batch size 模型执行日志进行分析整理，从而获得表 3.1 中的模型信息。

(1) 执行时间 (execute time)：在指定 batch size 下的计算节点的运行时间。Profiling 日志中计算节点执行结束时间与开始时间之差。为了获得测试 batch size 下的计算节点执行时间，方法根据 Profiling 日志中的多个 batch size 下的计算节点执行时间，使用线性回归模型^[66] (Linear Regression Model) 拟合测试 batch size 下的计算节点执行时间。

(2) 输出尺寸 (output size)：计算节点的输出张量所占用的内存空间之和。代表该计算节点可以进行优化的内存开销，由张量数据的维度和数据类型共同决定，其中，张量数据的维度为 $[N, H, W, C]$ ，分别代表输出数据的 batch size、长度、宽度和通道数。在长度、宽度和通道数固定的情况下，输出尺寸同 batch size 线性相关。

(3) 数据依赖 (data dependency)：计算节点之间的数据依赖关系。当计算节点 F_m 的输出是计算节点 F_n 的输入，则认为计算节点 F_n 依赖于计算节点 F_m 。方法遍历 Profiling 日志中收集的所有的计算节点，根据计算节点的输入输出关系构建数据依赖关系图。

REMEM 方法针对单个训练加速设备上的训练过程进行内存优化，设备信息是内存优化策略选择的重要依据。设备信息包括：设备的内存容量 (device memory) 以及主机和通信带宽 (bandwidth) 信息。其中，设备的内存容量即为自适应内存优化方法的优化目标，而通信带宽则决定了内存交换技术中的主机与训练加速设备之间的数据拷贝时间。

3.3.2 模型训练状态建模

在模型训练过程中，使用重计算或内存交换技术能有效节约内存资源，然而，重计算需要额外的计算资源，而内存交换技术依赖于主机与训练加速设备之间的通信资源，因此在训练过程中，为数据添加这些内存优化技术，会影响多种资源的使用状态。本小节提出了模型训练状态建模方法，该方法对训练过程中内存资源、计算资源和通信资源的使用状态进行建模，以明确数据应用不同内存优化技术对模型训练状态的影响，辅助策略搜索算法为数据选择合适的内存优化技术。

首先, 本文按照数据依赖关系对计算节点进行排序, 以拟合模型训练过程中的数据流动的拓扑顺序。模型的各个计算节点被视为图中的顶点, 而节点间的数据依赖关系则被视为有向边。假设模型中总共有 $2N$ 个计算节点, 那么这 $2N$ 个节点按照逻辑拓扑结构排序, 可以得到一个有序序列 $\{F_1, F_2, \dots, F_N, B_N, \dots, B_2, B_1\}$, 其中每个计算节点基于深度优先搜索 (Depth First Search, DFS) 进行排序, 仅在其所有依赖边的数据已经计算完成后才会被编号, 每个计算节点包含了序号, 输出特征数据内存大小, 计算代价等信息。

其次, 本文基于已经排序的计算节点序列信息, 将模型的训练情况建模成一个 $[3, N, N]$ 的状态矩阵 $S(S_{mem}, S_{com}, S_{comm})$, 分别代表了模型训练过程中的内存使用状态 S_{mem} , 数据计算状态 S_{com} 和数据通讯状态 S_{comm} 。第二维代表模型中的计算节点, 第三维则用于表示节点操作的逻辑时间。

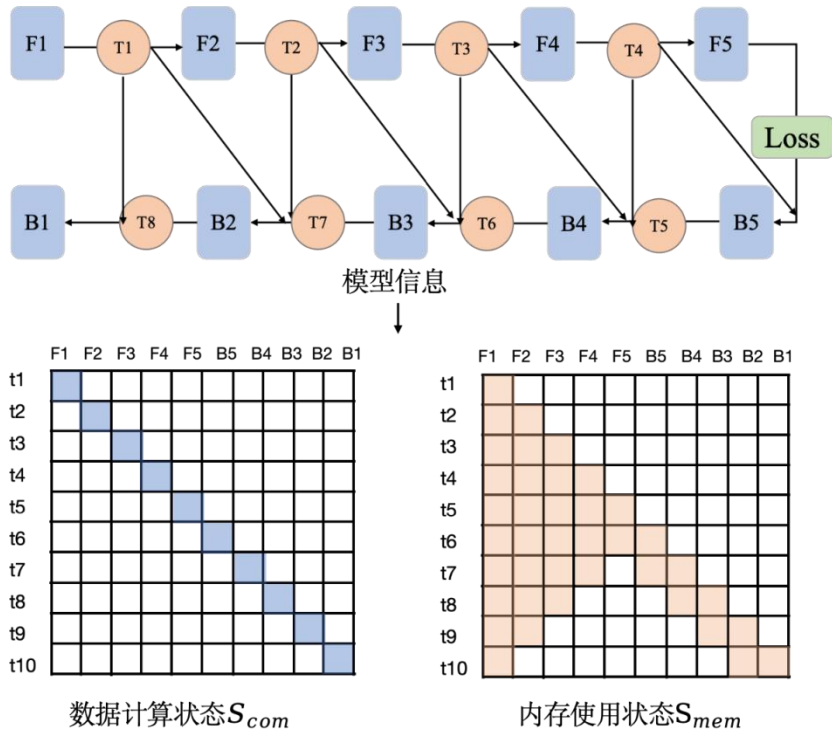


图 3.4 模型训练状态建模

然后, 根据特征映射数据序列初始化多维状态矩阵。内存使用状态 S_{mem} 用于记录数据驻留 GPU 内存中的时间和占用内存的大小, 如图 3.4 所示, 以一个 10 个节点的深度学习网络的训练过程为例, 模型中节点 F_1 的输出特征数据 T_1 需要作为节点 B_1 和 B_2 的输入, 那么从 F_1 执行前为 T_1 申请分配内存到 B_1 执行完毕释放内存, 也就是 $t \in [1, 10]$ 的逻辑时间段内, S_{mem} 的值都为数据 T_1 内存占用的大小; 数据计算状态 S_{com} 用于表示数据的计算过程, 初始状态下节点 F_i 的第 i 个时刻, S_{com} 中的对应值为节点 F_i 所需的计算时间; 数据通讯状态 S_{comm} 用于表示模型训练过程中的 GPU 和 CPU 主存之间的数据通信情况,

S_{comm} 中的值在初始状态时为 0。通过采用上述建模方法，本研究将模型训练过程中所需资源及时序信息表征为一个多维训练状态矩阵，从而为多种模型结构的优化策略搜索奠定了基础。

最后，随着内存优化策略的搜索过程，数据选择不同的内存优化策略，训练状态矩阵 S 中的数据也会随着策略选择变化，以模拟实际训练过程中的状态变化。

3.3.3 内存优化策略搜索空间建模

在进行模型训练过程的内存优化策略搜索时。定义策略搜索空间，明确搜索的范围，能加快搜索速度，确保算法能够有效地探索并优化策略。针对内存优化问题，本研究为每个待优化计算节点 F_i 的输出数据定义两个策略搜索空间：优化策略搜索空间 A_i 和策略执行时间间隔搜索空间 TC_i 。通过同时对数据优化的策略和策略执行时间进行搜索，实现对内存优化策略的精细控制。

首先，根据计算节点间的数据依赖关系确定可进行内存优化的计算节点。选择输出数据会被重复使用，且使用间隔可以支持数据优化策略执行的计算节点作为搜索节点集合 $F\{F_1, F_2, \dots, F_i\}$ 。具体来说，节点输出数据的两次相邻访问时间应当大于对该数据执行内存交换策略的时间（数据换出 GPU 内存和数据换入 GPU 内存所需的通信时间之和）以及重计算策略执行时间（该节点的计算时间）。

其次，为搜索节点 F_i 设置优化策略搜索空间 $A_i\{0,1,2\}$ 。搜索节点集合中每个 F_i 拥有以下优化策略选择范围：继续缓存在 GPU 内存（ $A_i = 0$ ），使用内存交换技术进行优化（ $A_i = 1$ ），使用重计算技术进行优化（ $A_i = 2$ ）。

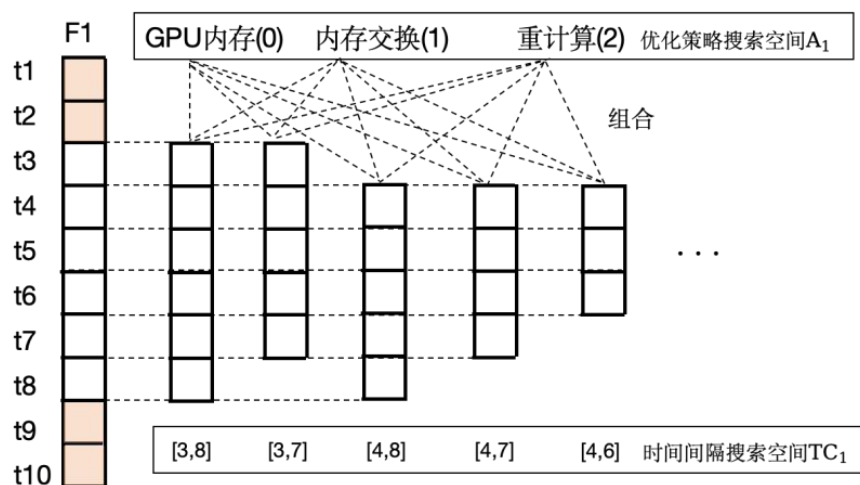


图 3.5 策略搜索空间示意图

然后，为搜索节点 F_i 设置策略执行时间间隔搜索空间 $TC_i\{c_1, c_2, \dots, c_i\}$ 。已知对于节点 F_i 的输出数据，可在数据第一次使用后立即开始内存优化措施，并在下一次需要使用数据之前完成数据的重新生成。然而，执行内存优化的时机对于内存优化效率具有显著影响，所以通过对数据优化的时机进行调整（延后数据的释放时间或提前数据的重新生成），以得出不同的数据优化时间窗口。

如图 3.5 所示，节点 F_1 的输出数据需要被 F_2 、 B_2 、 B_1 节点使用，这意味着 F_1 的输出数据会在 $t = \{2, 9, 10\}$ 被使用，而在 $t \in [3, 8]$ 的时间间隔内可以优化该数据占用的内存空间，通过调整内存优化技术的起止时刻，可以生成不同的策略执行时间间隔。同时，为了确保数据优化活动在一个合理的时间间隔内执行，方法引入了一个系数 k 来调节数据优化执行的最短时间间隔，特别当 $k = 1$ 时，对于给定节点 F_i 的输出数据，只存在一种优化时间间隔组合，即节点 F_i 的输出数据两次重用间的最长时间间隔。

$$\text{len}(c_i) \geq k * \max(\text{len}(c_i)) \quad (3.3)$$

最后，组合优化策略搜索空间 A_i 和策略执行时间间隔搜索空间 TC_i ，共同作为强化学习的内存优化策略搜索空间。

3.4 基于双智能体强化学习的内存优化策略搜索方法

在给定的模型结构和优化目标下，采用重计算和内存交换技术进行内存优化本质上是一个最优策略搜索问题。该问题的核心目标是如何为模型中的数据分配合适的内存优化策略，从而在有限内存资源的设备上完成模型训练的同时，最小化内存优化方法对模型训练效率的负面影响。为此，REMEM 引入了强化学习算法来探索模型训练过程中各种数据的最优内存优化策略。

基于 3.3 节中构建的强化学习训练环境，本文使用双智能体策略搜索算法搜索模型中各特征映射数据的内存优化策略和策略执行时机。3.4.2 节中的策略模拟执行算法模拟了内存优化策略选择在真实训练环境的执行结果，3.4.3 节中的策略反馈优化算法通过模拟执行结果计算反馈值，优化强化学习智能体的策略选择。具体的搜索步骤将在下文中进行详细介绍。

3.4.1 双智能体策略搜索算法

围绕内存优化问题，本文采用双智能体强化学习方法对搜索节点集合 $F\{F_1, F_2, \dots, F_i\}$ 的内存优化策略 $A\{A_1, A_2, \dots, A_i\}$ 和策略执行时间间隔 $TC\{TC_1, TC_2, \dots, TC_i\}$ 进行搜索。搜索算法的输入即为 3.3 节中的拟合模型

训练状态建模的环境 $S(S_{mem}, S_{com}, S_{comm})$, 搜索节点集合 F 以及策略搜索空间 $\{A, TC\}$ 。搜索算法的输出则是对数据节点 F_i 所选择的优化策略序列和优化策略执行时间间隔序列。

首先, 初始化两个强化学习智能体 `action_agent` 和 `time_agent` 分别用于搜索优化策略和策略执行时间。智能体基于值函数思想的 DQN 算法进行实现, 每个智能体中包含策略网络 Q 和学习网络 Q_{learn} , 均为两个隐层和一个全联接层组成的前馈神经网络,

其次, 策略网络 Q 接收当前的模型训练状态 S_i 作为输入, 输出该训练状态下可选择策略序列的预测奖励值 $Q(S_i, A_i)$ 和 $Q(S_i, TC_i)$ 。

然后, 智能体 `action_agent` 选择当前训练状态下预测奖励值最大的策略 $\max_{A_i} Q(S_i, A_i)$ 作为节点 F_i 所选择的内存优化策略。智能体 `time_agent` 中则会根据当前训练状态以及 `action_agent` 策略选择结果情况来筛选策略执行时间间隔, 具体来说: 当策略选择将数据继续缓存在 GPU 内存中时, 则无需选择特定的优化时间间隔; 当策略选择使用重计算技术进行数据优化时, 则仅需搜索数据重新计算所需的时间点; 当策略选择使用内存交换技术来减少内存消耗时, 方法会选择当前模型训练状态状态下预测奖励值最大的内存优化时间间隔 $\max_{TC_i} Q(S_i, TC_i)$ 。同时, 为了进一步优化智能体的搜索方法, REMEM 方法在智能体进行策略选择的过程中添加了随机扰动。即在使用智能体进行策略选择的过程中, 不总是选择预测模型中输出概率最大的策略, 而是以一定的概率随机选择一个策略并返回。通过这样的方式, 能够扩大智能体训练记录的样本多样性, 从而提升智能体的训练和搜索效果。

最后, 返回强化学习智能体为数据节点 F_i 选择的优化策略 A_i 和 TC_i 用于更新模型训练状态。

3.4.2 策略模拟执行算法

在使用强化学习进行策略搜索的过程中, 策略优化的核心机制依赖于从环境中动态获取策略选择的反馈, 本研究方法针对基于强化学习的内存优化策略搜索问题构建了一个环境模型, 该模型采用多维状态矩阵表示模型训练状态。在此框架中, 数据优化策略的选择过程伴随着模型训练状态的动态更新, 从而确保策略选择能够根据最新的环境状态进行调整和优化。

状态矩阵更新算法基于数据节点 F_i 选定优化策略 A_i 和策略执行时间时间间隔 TC_i 来更新模型训练状态 S_i , 更新规则如下:

- (1) 当 $A_i = 0$, 即策略选择将节点数据继续缓存在 GPU 内存中时, 模型

训练过程中的内存及计算通信状态并未发生改变，不对模型训练状态进行更新。

(2) 当 $A_i = 1$ ，即策略选择使用内存交换对节点数据进行优化时，需要将数据先换出到 CPU 内存，再换回 GPU 内存。首先在通信状态矩阵 S_{comm} 中，策略优化的起始时刻 ($TC_i.start$) 和结束时刻 ($TC_i.end$) 添加数据通信代价， F_i 的数据通信代价 $swapCost$ 由节点数据尺寸除以通信带宽得出。其次，在 $TC_i.start$ 到 $TC_i.end$ 之间，因为数据已经换出到 CPU 内存中，将内存状态矩阵 S_{mem} 中对应时间段的内存消耗设置为 0。

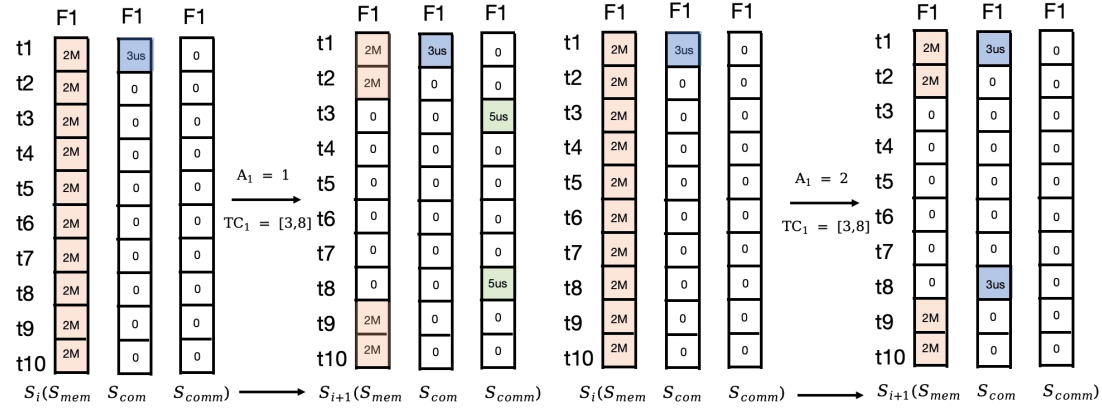


图 3.6 状态矩阵更新示意图

(3) 当 $A_i = 2$ ，即策略选择使用重计算对节点数据进行内存优化时，需要在节点数据被使用前重新计算节点 F_i ，在计算状态矩阵 S_{com} 上，在策略优化的结束时刻 ($TC_i.end$) 添加 F_i 的重计算成本，并且在数据使用完成到 $TC_i.end$ 之间，因为数据分配的内存空间已释放，内存状态矩阵中对应时间段的内存消耗设置为 0。特别的，重计算成本 $recomCost$ 的计算机制需详细说明：若数据需重新计算，则生成该数据所需的输入数据必须已存于 GPU 内存之中。通过对时间间隔的搜索，目标是确保在进行数据重计算以生成所需数据时，所有依赖的输入数据已被重新计算并生成。若模型所依赖的数据不存于 GPU 内存中，则将这些依赖数据的计算时间累积至当前数据的计算时间，作为重计算成本。如果存在链式依赖的输入数据均不在 GPU 内存中，则将这些链式依赖的计算时间全部累加至当前数据的重计算成本中。

算法 3.1: 状态矩阵更新算法

Input: S_i, F_i, A_i, TC_i

Output: S_{i+1}

- 1: if $A_i = 0$ then //缓存在 GPU 中
- 2: return
- 3: else if $A_i = 1$ then //选择内存交换技术
- 4: $S_{comm}[F_i.nodeId][TC_i.start] = F_i.swapCost$

```

5:  $S_{comm}[F_i.nodeId][TC_i.end] = F_i.swapCost$ 
6: for  $t \in TC_i$  do
7:    $S_{mem}[F_i.nodeId][t] = 0$ 
8: end for
9: else if  $A_i == 2$  then //选择重计算技术
10:  $S_{com}[F_i.nodeId][TC_i.end] = F_i.recomCost$ 
11: for  $t \in TC_i$  do
12:    $S_{mem}[F_i.nodeId][t] = 0$ 
13: end for
14: end if
15: return  $S$ 

```

3.4.3 策略反馈优化算法

在使用固定 batch size 进行模型训练过程中，内存使用峰值 (peak_memory) 指使用该 batch size 进行训练所需的最大内存量，模型训练时间 (execution_time) 则是指模型对一个 batch 的样本数据进行训练的平均时间。优秀的内存优化策略能在降低内存使用峰值的同时减少对模型训练时间影响，因此，在本小节提出的策略反馈优化算法中，将综合内存使用峰值和模型训练时间计算内存优化策略的反馈值，根据该反馈值更新强化学习搜索智能体。

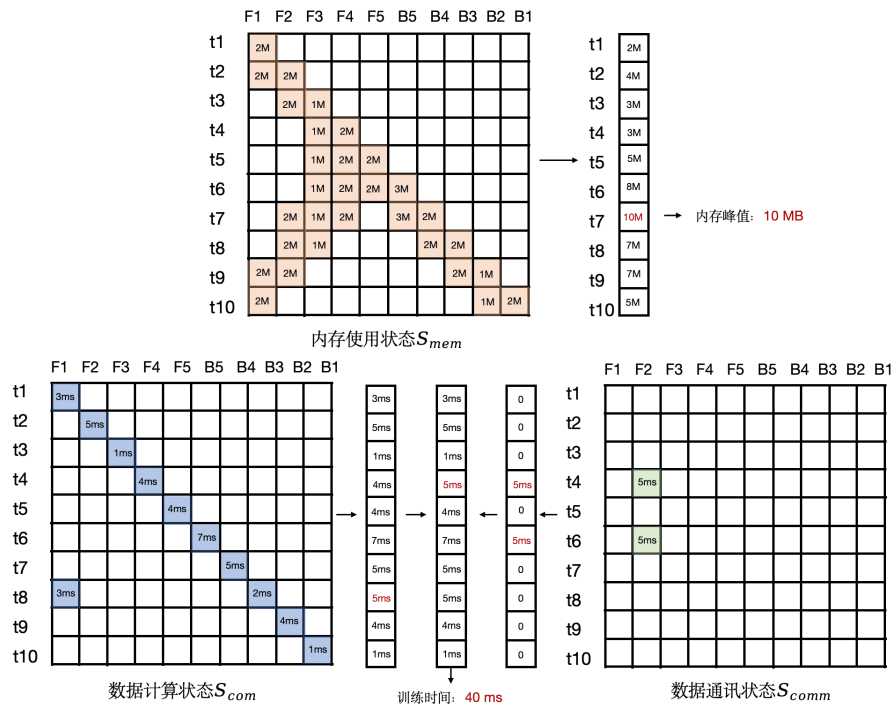


图 3.7 内存峰值及模型训练时间计算过程示意图

首先，基于更新后的训练状态矩阵 S_{i+1} 分析模型训练过程中的内存使用峰值 $peak_memory_{i+1}$ 和模型训练时间 $execution_time_{i+1}$ 。如图 3.7 所示，首先，将内存状态矩阵沿时间维度进行累加，整合成一维数组。在此数组中，位于第 t 个位置的值代表了模型在时间点 t 的总内存占用量，其中，数组中的最大值即为当前模型训练状态下的内存使用峰值。当内存峰值小于当前训练设备的可用内存容量 $device_memory$ 时，可视为本轮优化目标已经达成，随后的数据节点无需再选择内存优化策略。

$$Done = peak_memory_{i+1} \leq device_memory \quad (3.4)$$

不同于模型训练过程中内存占用的计算方法，模型训练时间的计算规则则相对复杂，这是因为要同时考虑到模型内存交换和重计算方法带来的额外时间代价。首先，由于正常计算和重计算的过程中均需占用 GPU 计算资源，而这些计算活动的时间不能重叠。因此，通过沿时间维度累加合并计算状态矩阵，可以有效模拟重计算优化对模型训练时间所带来的代价。其次，虽然内存交换策略中的数据传输时间可以与模型的正常计算时间重叠，但由于数据传输过程需独占总线通道，当多个传输任务同时发生时，便需排队等待。因此，本方法通过累加并合并通信状态矩阵，与同一时间点的计算时间进行对比，从而整合无法与计算重叠的数据通信时间。通过汇总这些时间消耗，便可得出模型训练过程所需的总时间。

$$r1 = mem_saving / max(extra_execution_time, 1) \quad (3.5)$$

$$r2 = mem_saving / opt_execution_time \quad (3.6)$$

$$flag = \begin{cases} -1 & A_i = 0 \text{ and } !Done \\ 1 & (A_i = 1 \text{ or } A_i = 2) \text{ and } !Done \end{cases} \quad (3.7)$$

$$R = flag * (r1 - r2) \quad (3.8)$$

其次，记录每次模型训练状态更新前后的内存使用峰值 $peak_memory_i$ 和模型训练时间 $exection_time_i$ ，用于计算对应策略选择的奖励值。 $r1$ 代表了节约内存空间所花费的代价，其中， mem_saving 表示当前数据的策略选择能够减少的最大内存使用量，该值等于执行策略选择后的节约的内存空间 $(peak_memory_{i+1} - peak_memory_i)$ ， $extra_execution_time$ 代表策略选择带来的额外训练时间，即策略执行前后的训练时间差额 $(execution_time_{i+1} - execution_time_i)$ 。 $r2$ 用于表示节约内存空间所带来的价值，式 (3.6) 中的 $opt_exection_time$ 是指策略实际执行优化的训练时间，即策略优化的起始时刻 $(TC_i.start)$ 和结束时刻 $(TC_i.end)$ 之间的模型训练

时间, 即拓扑序号在 $TC_i.start$ 和 $TC_i.end$ 之间的所有计算节点执行时间之和。显然, 在进行搜索的过程中期望策略选择带来的额外执行时间少且优化的时间间隔长, 即 $r1$ 的值越大越好, 而 $r2$ 的值越小越好。此外, 在内存优化目标未达成的情况下, 选择让数据驻留在 GPU 内存中被视为次优策略, 因此, 通过一个标志变量 $flag$ 来控制策略选择的奖励值的正负。

最后, REMEM 方法基于 DQN 算法中的经验回放和固定 Q 目标技术对智能体 `action_agent` 和 `time_agent` 进行训练和更新。具体来说, 每次搜索状态 S_i , 策略选择 $\{A_i, TC_i\}$ 以及由 3.4.2 节方法所更新的状态 S_{i+1} 以及计算的奖励值 R_i 都会加入经验回放池, 从经验回放池中随机选择数据来进行学习能打破数据之间的关联性。固定 Q 目标技术通过两个相同的结构的网络提高强化学习训练的稳定性: 一个策略网络 Q 用于根据当前状态预测奖励值 $Q(S_i, A_i)/Q(S_i, TC_i)$, 另一个学习网络 Q_{learn} 根据经验回放池中的数据来学习并更新模型参数, 并在固定周期后同步更新预测模型的参数, 通过多轮迭代, 模型预测的奖励值会更偏向于实际训练过程中的策略奖励值, 从而智能体能够根据训练状态选择出在收益更大的策略。

3.5 REMEM 方法实现

本文提出了基于双智能体强化学习的自适应内存优化方法, 该方法在确定的设备和模型结构基础上, 根据测试的训练样本 `batch size`, 搜索一组内存优化策略实现模型的高效训练过程, 方法的具体实现流程如下:

(1) 在设备上运行多个不同 `batch size` 的模型训练过程, 基于 Profiling 技术收集模型中各计算节点的数据依赖关系, 以及模型训练信息包括节点计算时间和输出尺寸, 根据收集信息计算测试 `batch size` 下的计算时间和输出尺寸。

(2) 使用模型信息初始化多维状态矩阵 $S_0(S_{mem}, S_{com}, S_{comm})$, 用于表示未进行内存优化情况下的深度学习训练过程的内存资源, 计算资源和通信资源的使用状态。

(3) 基于模型信息确定可进行内存优化的搜索节点集合 $F\{F_1, F_2, \dots, F_i\}$, 约束计算节点 F_i 可选择的内存优化策略 A_i , 和策略执行时间间隔 TC_i 。

(4) 初始化智能体 `action_agent` 和 `time_agent`。

(5) 遍历搜索节点集合 F , 并基于当前的训练状态 S_i , 通过 `action_agent` 选择数据节点 F_i 的内存优化策略 A_i , 通过 `time_agent` 选择节点 F_i 的策略执行时间间隔 TC_i 。

(6) 根据选择的内存优化策略 A_i 和策略执行时间间隔 TC_i 更新模型训练状

态矩阵 ($S_i \rightarrow S_{i+1}$)，以模拟真实训练环境下优化策略执行效果。

(7) 基于更新后的模型训练状态矩阵 S_{i+1} ，计算当前策略选择的反馈值 R_i ，根据反馈值更新智能体 `action_agent` 和 `time_agent`。

(8) 重复执行步骤 (5-7)，记录最优的模型优化策略组合 $\{A, TC\}$ ，在真实环境中执行最优内存优化策略。

算法 3.2: REMEM 方法

Input: model (模型), device (设备), batch size, round_num

Output: 最优内存优化策略

```

1: model_info = profiling(model, batch size)
2: S = init_state(model_info) //模型训练状态建模
3: F, A, TC = init_search_space(model_info) //策略搜索空间建模
4: action_agent.init() //初始化 action_agent
5: time_agent.init() //初始化 time_agent
6: for round < round_num do
7:   R = [], round_A = [], round_TC = []
8:   for  $F_i \in F$  do //遍历待优化节点
9:      $A_i$  = action_agent.choose(S) //选择数据的内存优化策略
10:     $TC_i$  = action_agent.choose( $A_i$ , S) //选择优化策略执行时间间隔
11:    updateState(S) //策略模拟执行算法
12:     $R_i$  = step(S,  $F_i$ ,  $A_i$ ,  $TC_i$ ) //策略反馈值计算
13:    action_agent.learn() //智能体更新
14:    time_agent.learn() //智能体更新
15:   if Done(S, device) then //判断是否完成优化目标
16:     Break
17:   end if
18: end for
19: record(R, round_A, round_TC) //记录最优内存优化策略
20: end for

```

REMEM 方法根据搜索的最优内存优化策略，在现有 AI 训练框架的基础上，对模型实际训练过程中数据的释放和加载时机进行了调整。以 TensorFlow 框架为例，TensorFlow 框架的图模式将模型计算过程表示为一个计算图，图的节点代表操作，而图的边代表在操作之间流动的数据张量，在训练时按照图执行实际计算过程，方法通过修改计算图来指导模型训练中的内存优化过程。为

为了实现重计算优化，方法在计算图中新增了需要重计算节点的副本，同时为了避免优化节点立即执行，通过新增触发器节点的方法来控制优化节点的执行时机。为了实现内存交换优化，REMEM方法使用了 TensorFlow 框架中提供了两个异步通信节点：CopyFromGpuToHost 和 CopyFromHostToGpu，分别负责数据换出和数据换入操作，这两个节点通过独立的 Cuda Stream 来进行数据拷贝工作，从而与当前的计算流进行重叠。此外，TensorFlow 中的张量引用计数机制被用来管理 GPU 内存的释放，数据张量在进行内存分配过程中会同时新建引用计数器，当引用计数等于零时，数据所关联的 GPU 内存会被释放。在执行优化后的计算图时，进行内存交换操作的特征数据在换出操作结束之后，它和计算图中其他节点将不存在依赖关系，类似的，使用重计算技术优化的特征数据在断开与反向传播对应节点的依赖关系之后，该数据也将释放其所占用的内存空间，从而减少训练过程中的内存需求。

3.6 本章小结

本章介绍了基于双智能体强化学习的自适应内存优化方法 REMEM，该方法为神经网络训练过程中产生的数据搜索分配合适的内存优化策略，方法构建拟合神经网络训练过程的多维状态矩阵，并基于神经网络模型结构确定内存优化策略空间；采用双智能体强化学习的方法协同进行策略搜索；根据策略搜索结果的奖励值，通过多轮迭代优化以得到收益最大的策略搜索结果，在达到计算加速设备内存优化目标的同时，避免密集化的通信和计算资源消耗，从而使得单个训练加速设备可以处理更大规模的神经网络模型或者更大 batch size 的数据样本。

第四章 基于 AI 训练任务访存特征的内存分配算法

4.1 问题描述

在深度学习领域，模型的规模和复杂度与日俱增，而这种增长往往伴随着对内存资源的巨大需求。然而，训练设备的物理内存容量是有限的，这导致了内存资源的分配和管理成为了性能优化的关键瓶颈之一。传统的内存分配机制，如直接使用操作系统或硬件默认的内存管理器（例如，通过 `cudaMalloc` 在 CUDA 环境中分配内存），这些内存管理器往往采用简单的分配策略，这在处理大规模或动态变化的内存请求时，会导致内存碎片化严重、分配延迟增加、以及资源利用率低下等问题。这些问题不仅影响了计算任务的执行效率，还可能导致内存资源的浪费，从而限制了模型规模的扩展和训练任务的执行能力。TensorFlow 和 PyTorch 中的内存分配算法管理多个内存块，并根据内存请求的尺寸复用不同的内存块，从而提高分配速度。此外，内存分配算法还通过切分和合并方法动态地调整内存块的大小和数量，进一步提高内存使用效率和应用性能。因此，内存分配算法成为了解决当前训练资源需求和设备内存之间的不平衡问题，实现高效内存管理的关键技术之一。

本文观察到，现有内存分配算法在进行数据分配的过程中，会产生大量内存碎片，包括内部碎片和外部碎片。内部碎片发生在分配给程序的内存块内部，如图 4.1 中 `Chunk1` 所示，当分配器给 $Request_i$ 分配了 $Chunk_i$ 用于管理该请求的内存，则会产生 $Chunk_i.size - Request_i.size$ 大小的内部碎片。这部分未被有效利用的内存资源，虽然仍然归程序所有，却未被实际使用，导致内存利用率降低。与此相对的是外部碎片，当 $Chunk_i$ 未被使用，且 $Chunk_{i-1}$ 和 $Chunk_{i+1}$ 均已被分配的情况下， $Chunk_i$ 即形成了外部碎片。如图 4.1 中的 `Chunk2`，它散布于被分配的内存块之间，无法通过合并方式改变其尺寸，使得大量小的内存碎片分散在整个内存空间中。尽管总的空闲内存量足以满足某些内存请求，但由于这些空闲内存不连续，从而导致分配失败或效率下降。

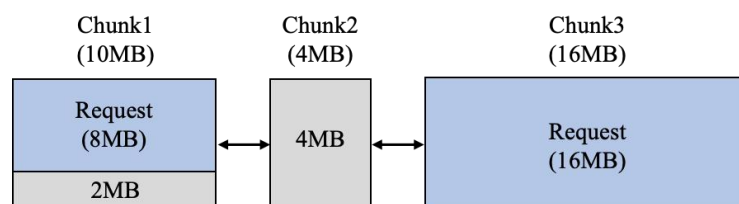


图 4.1 内存碎片示意图

现有内存分配算法忽略了模型训练过程中访存请求规律。如图 4.2 所示, 本文收集了 VGG16^[64]模型五轮训练过程中的内存使用情况。从图像中可以观察到在模型的训练过程中, 除了在第一轮训练中, 需要初始化模型的权重和相关参数以及缓存部分计算结果, 后续的内存分配请求随着训练迭代呈现规律的周期特性。并且, 从访存请求的视角分析每轮训练中的数据请求, 训练的初期主要以内存分配请求为主, 对应模型前向传播阶段计算并存储特征映射数据的需求。而在反向传播阶段, 内存请求既包括数据的分配也涉及到数据的释放, 这反映了在逐层计算梯度时为梯度数据申请空间以及计算完成后特征映射数据的释放过程。一轮训练结束后, 所涉及的数据除模型参数外全部释放, 以便于内存空间的再次利用。同时, 由于每轮训练结束时参数数据所占用的内存空间不会释放, 未经整理的参数数据散布在内存空间中, 分割了完整的内存空间, 导致大量无法合并的外部碎片。并且在模型训练过程中, 存在许多大尺寸的临时数据请求, 对这些数据释放的空间进行再分配时, 容易形成内部碎片, 造成资源浪费。

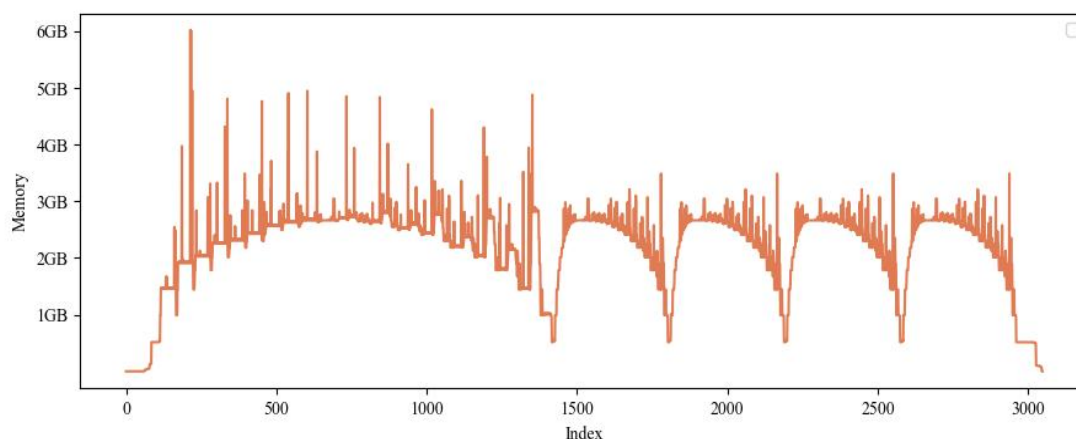


图 4.2 VGG16 模型多轮训练过程中的内存使用情况

综上, 本文总结了现有深度学习模型训练场景下的内存分配算法存在的问题:

(1) 如何减少数据分配过程中产生的内存碎片? 在模型训练过程中, 内存分配与释放请求的交错执行, 邻近请求尺寸存在显著差异, 导致顺序化执行内存分配和释放请求的过程中会产生大量内存碎片, 造成的内存资源浪费的情况。因此需要制定高效的内存分配算法来最小化内存碎片, 优化内存资源的使用效率。

(2) 如何高效利用模型训练过程中的访存请求规律? 现有内存分配算法只考虑了内存分配请求的尺寸, 而模型训练过程中的内存分配请求具有典型的周期迭代特性, 并且每个训练周期中都包含了大量的临时请求数据, 通过识别并利用模型训练过程中的访存规律, 能够更加合理地分配内存空间, 从而减少对

硬件资源的需求，降低训练成本。

针对上述问题，本文围绕 AI 训练任务访存特征提出了 MARSP(Memory Allocator based on Request Size and Period)内存请求分配算法。

4.2 MARSP 方法概述

面向深度学习模型过程中的访存请求特征，本文提出了基于 AI 训练任务访存请求特征的内存分配算法，MARSP(Memory Allocator based on Request Size and Period)。方法的主要流程如图 4.3 所示。

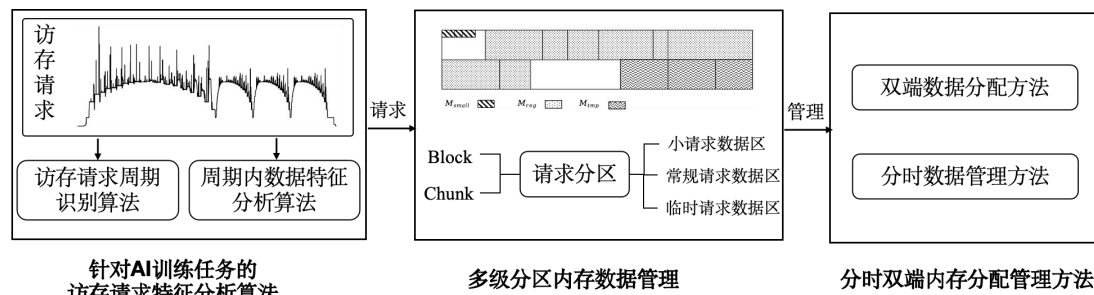


图 4.3 MARSP 流程图

首先，提出针对 AI 训练任务的内存请求特征分析算法。算法使用滑动窗口结合距离差分的方法对访问请求的规律进行分析，识别训练周期的起始时间和周期长度。并基于周期内的数据访存特征，识别训练过程中会被快速释放的临时数据。

其次，采用基于多级分区的内存数据管理，以优化数据管理结构。使用固定尺寸的 Block 和动态尺寸的 Chunk 结构归纳数据请求。基于访问请求的尺寸信息将内存空间分为两个 Block 区域，并进一步根据周期特征将内存空间细分为小请求数据区 M_{small} ，常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} ，分别管理训练过程中具有对应特征的数据。

最后，提出分时双端内存管理分配方法，根据请求的周期特征分别使用内存整理及内存合并两种技术分时管理数据区 M_{reg} 。针对周期内临时请求数据存放在高地址的临时请求数据区 M_{tmp} ，周期内持久数据则存放在低地址的请求数据区 M_{reg} ，以减少内存碎片的产生。

下面，将对 MARSP 方法进行详细介绍。

4.3 针对 AI 训练任务的访存请求特征分析算法

在深度学习模型训练过程中，模型通过一系列迭代过程逐步优化参数，每

次迭代均涉及前向传播、梯度计算和反向传播等步骤，而这些步骤对内存的需求具有明显的周期性变化。这使得 AI 训练任务的访存请求也具有典型的周期规律，如何准确地识别该特征，进而根据模型训练过程中的访存请求规律有效调配和管理内存资源，是一个重要的命题。针对该问题，本小节将介绍访存请求周期识别算法和请求特征分析算法，这些算法通过深入分析模型训练过程中访存请求的周期特性和周期内的数据请求特征，动态识别请求的访问模式，为内存的分配管理方法提供更为准确的决策支持，从而有效提高内存资源的使用效率。

4.3.1 访存请求周期识别算法

观察模型的多轮训练过程中的访存请求可知，在模型的初始训练阶段，由于数据加载、参数初始化等流程，其内存分配请求序列有别与后续周期性请求序列，呈现序列周期长，序列中分配请求密集的情况，而在此后的多轮训练过程中访存请求按照周期重复，且周期长度和周期内的访存请求取决于模型的结构和训练参数设置。因此，采用算法动态识别区分请求周期，有助于对内存空间的合理布局，实现高效的内存管理。针对模型训练过程中内存请求，本文基于滑动窗口^[67]和距离差分思想提出内存请求周期识别算法。

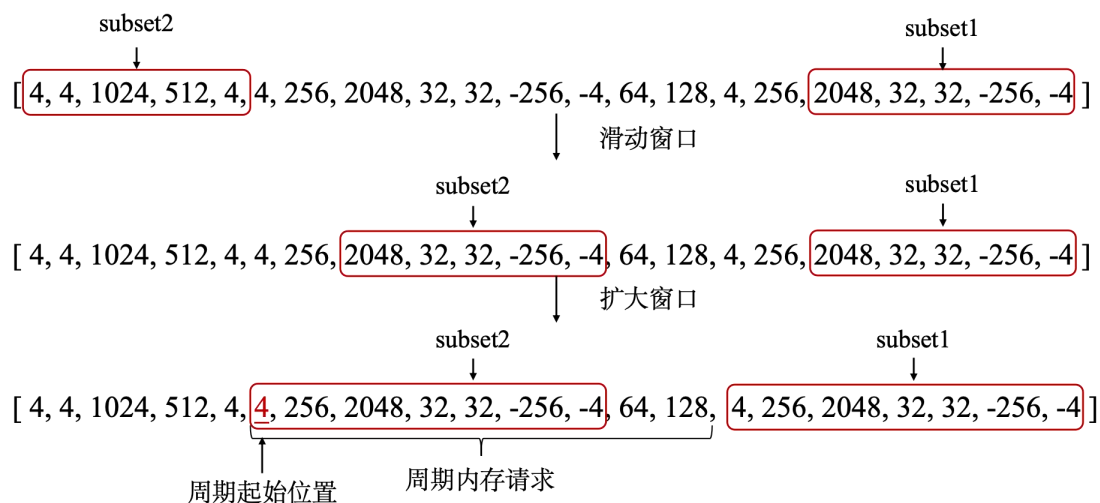


图 4.4 请求序列比较窗口示意图

MARSP 方法在管理分配内存请求的同时将请求信息记录为一个有序序列，并基于该请求序列寻找内存请求的周期规律。如算法 4.1 所示，算法输入为内存请求序列，序列中的每个请求包含了数据分配/释放标识，请求尺寸，请求分配地址信息，算法的输出为识别到的周期长度和起始下标。此过程利用 *max_period_length* 作为滑动窗口的初始大小，并据此调整算法执行频率，同

时以 min_period_length 为窗口最小尺寸限制，当窗口长度小于 min_period_length 停止周期识别过程。

算法首先截取两个子序列： $subset1$ 从请求序列末尾向前， $subset2$ 从请求序列开始向后，初始窗口大小设为 max_period_length 。接着，使用距离差分方法比较序列 $subset1$ 和序列 $subset2$ 之间的差异。如式 (4.1) 所示，依次比较 $subset1$ 和 $subset2$ 中的请求数据。如果 $subset1$ 和 $subset2$ 之间的距离差分不为零， $subset2$ 向后滑动继续进行比较；如果距离差分为零，则表示两个序列中的请求数据相等，即这两个序列位于周期中的相同位置。此时， $subset1$ 和 $subset2$ 同时扩大比较窗口来寻找周期起始位置，当两子序列间的距离差分再次非零时，则表示 $subset2$ 序列头即为周期的起始位置，而 $subset1$ 序列头和 $subset2$ 序列头之间的数据即为一个模型训练周期中的内存请求数据。

$$distance = \sum_{i \in subset} |subset1_i - subset2_i| \quad (4.1)$$

算法 4.1: 周期识别算法

Input: requests, max_period_length, min_period_length

Output: period_start_index, period_length

```

1: period_start_index = -1, period_length = -1
2: search_length = max_period_length
3: while search_length > min_period_length do
4:   subset1 = requests[-search_length : ]
5:   for start_index  $\in$  [0, len(requests)-search_length] do //滑动比较窗口
6:     subset2 = requests[start_index : start_index+search_length]
7:     distance = compare_distance(subset1, subset2) //计算距离差分
8:     if distance != 0 then
9:       continue
10:    end if
11:    slide = 0
12:    while start_index-slide >= 0 do //扩大比较窗口
13:      subset1 = requests[-(search_length + slide) : ]
14:      subset2 = requests[start_index - slide : start_index + search_length]
15:      distance = compare_distance(subset1, subset2) //计算距离差分
16:      if distance != 0 then
17:        break

```

```

18:   end if
19:   period_start_index = start_index - slide //计算周期起始下标
20:   period_length = len(requests) - search_length - start_index //计算周期长度
21:   end while
22: end for
23: search_length = search_length - min_period_length//缩小比较窗口
24: end while
25: return period_start_index, period_length

```

最后，在无法找到完全匹配的序列时，缩小窗口的尺寸进行更细粒度的周期识别，直到窗口尺寸小于 min_period_length 时停止该轮搜索识别过程。

4.3.2 周期内请求特征分析算法

利用访存请求周期识别算法，可以精确捕获模型训练过程中的周期性请求及其数据信息。这些周期性请求中反映了模型训练的各个阶段，包括前向传播和后向传播等过程。而对模型单轮训练过程，即一个周期内的请求数据进行分析，可以观察到周期内存在大量内存使用量快速变化的情况。如图 4.5 所示，该图由模型训练过程中的内存分配和释放请求按序累加生成，在训练周期内的前向传播和后向传播过程中，频繁出现了临时数据的分配和释放请求，这类请求不仅空间占用大，并且分配与释放的时间间隔短，释放的空间在进行再分配的过程中易导致内存碎片的产生。因此需要分析周期内的数据请求，识别具有临时特征的数据请求。

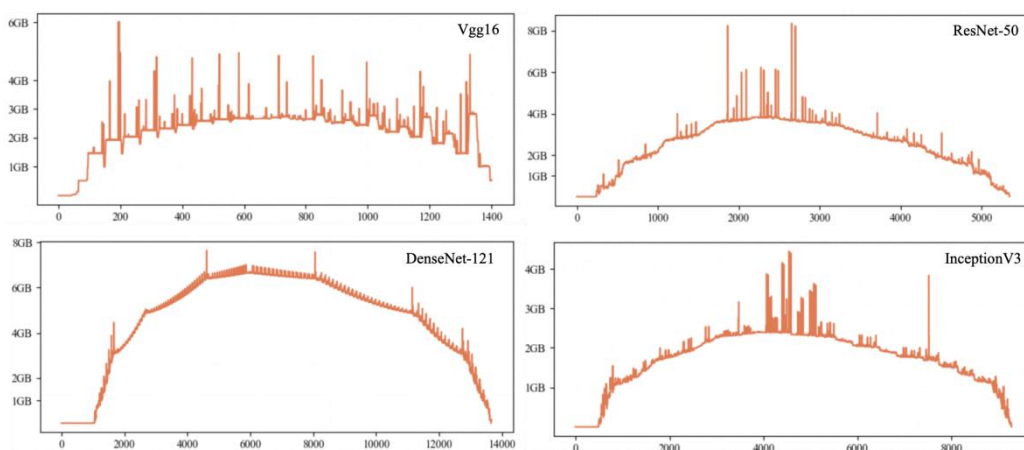


图 4.5 模型单轮训练过程中的内存使用情况

为了有效处理周期内的临时数据分配请求，本研究提出了一种周期内请求特征分析算法。具体来说，遍历周期性内存请求，设置周期内的请求起始索引为 0，对相对索引为 i 的分配请求，将该请求同索引为 $(i, i + tmp_slot]$ 之间的

释放请求进行比对，如果存在一个与当前分配请求地址相同的释放请求，说明该分配请求对应的数据是训练过程中产生的临时数据，会在短时间内释放。通过这种方法，一旦识别出这类临时数据请求，算法便会记录这些数据在模型训练周期中的相对索引位置 i 。的这一记录不仅有助于在数据分配管理流程中准确标识临时数据，同时为优化内存资源分配提供了重要依据。

在实际进行数据分配的过程中，会根据当前请求在周期内的索引进行判断，如果是临时数据请求，则由单独的区域为管理该请求，以避免临时请求在内存空间中留下无法利用的内存碎片。

4.4 多级分区的内存数据管理

针对模型训练过程中产生的内存碎片问题，在内存数据管理上，使用多种数据结构来存放内存数据，并结合请求特征和内存分区管理的思想，提出基于多级分区的内存数据管理。

在进行内存管理过程中，Block 和 Chunk 是两种常用的数据结构。其中，Block 通常是大小固定的内存块，用于向设备一次性申请或者释放固定尺寸的内存空间，相对地，Chunk 的设计允许其动态调整大小，从而为内存管理提供更细粒度的控制。在 MARSP 方法的数据结构设计中，同时使用 Block 和 Chunk 两种数据结构来共同管理内存空间。其中，Block 作为第一级数据管理结构，用来预分配固定尺寸的内存空间，而包含在 Block 内的多个 Chunk 作为第二级数据管理结构，负责对这些预申请的内存进行进一步的细分，以满足不同大小和生命周期的内存分配需求。

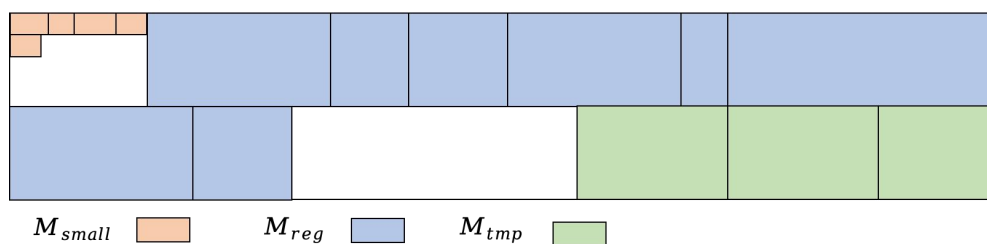


图 4.6 多级分区数据结构

进一步，基于 Block 和 Chunk 数据结构，将申请的设备内存分为两个固定尺寸的 Block，Block1 中包含小请求数据区 M_{small} ，Block2 中包含常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} ，三个区域都由多个 Chunk 组成，区域内的 Chunk 通过双向链表结构互相连接。如图 4.6 所示，首先，系统向设备申请固定尺寸的内存空间作为分配算法自行管理的内存区域，并在内存区域中的起始位置预留一块固定大小的 Block1，用于管理小请求数据区 M_{small} ，在区域内部通过多个 Chunk 块来管理小请求数据的分配和释放请求。常规请求数据区

M_{reg} 和临时请求数据区 M_{tmp} 则由 Block2 进行管理。常规请求数据区 M_{reg} 与 M_{small} 相邻，位于 Block2 的头部，由多个非固定大小的 Chunk 块组成。临时请求数据区 M_{tmp} 被分配在 Block2 的末端，与 M_{reg} 类似，该区域同样由多个动态大小的 Chunk 构成。

其中，小请求数据区 M_{small} 专门用于处理较小尺寸的内存请求，防止多个小块数据分割了连续的内存空间，造成内存碎片，导致后续内存分配请求无法满足。常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} 则分别管理模型训练过程中具有不同特征的访存请求。常规请求数据区 M_{reg} 旨在管理非周期数据性数据请求，以及周期请求中在缓存中存放时间较长的请求数据，如模型训练过程中的特征映射数据等。临时请求数据区 M_{tmp} 用于处理周期性请求中的会被快速释放的数据，如训练过程中卷积层申请的临时空间以及进行内存优化的数据等。

4.5 分时双端内存分配管理方法

随着深度学习技术的快速发展，模型训练过程中设备内存资源的紧张状况日益凸显，这使得研究如何降低模型训练的内存消耗以及如何高效管理设备内存资源这一命题成为了紧迫需求。在模型训练过程中涉及到频繁的内存分配和释放申请，而现有的内存分配算法在这一场景下往往不会考虑模型训练过程中的内存请求特性，这会产生大量内存碎片，浪费内存资源。本研究提出一种分时双端内存分配管理方法，该方法根据内存请求的周期特性，在不同时刻应用不同的技术手段管理内存空间。同时，针对周期内的训练数据，研究采用了双端数据分配方法分别对模型训练过程中临时数据和非临时数据进行管理，从而更精细地控制内存资源的分配与释放，实现内存资源的高效配置和使用。

4.5.1 分时数据管理方法

在模型训练的过程中，首轮训练通常涉及到大量的数据初始化与缓存过程，这与后续周期性的内存请求在性质上有显著差异。并且，在 MARSP 方法中采用的内存请求规律分析算法为一种动态算法，该算法在处理内存请求的同时，分析内存请求的规律。因此，本方法将内存访问请求分为三个阶段：初始化阶段、规律分析阶段及周期性请求阶段。并且在初始化及规律分析阶段，采用内存整理技术以最小化内存占用；在周期性访问阶段，通过内存合并技术实现内存的高效复用。

通过对模型训练过程中内存使用情况进行分析，可以观察到：在初始化阶

段, 对内存的需求最大, 但是在实际训练的过程中, 却会在规律分析阶段出现因为内存不足无法继续训练的情况, 这是由于在初始化阶段结束后, 内存请求占用的空间虽然释放了, 然而在数据释放过程中内存空间中产生了大量内存碎片, 导致新的内存分配请求无法满足的情况。针对这种现象, 分时数据管理方法提出使用内存整理技术管理初始化阶段和规律分析阶段的访存请求, 使用内存合并技术管理周期性请求阶段的访存请求。

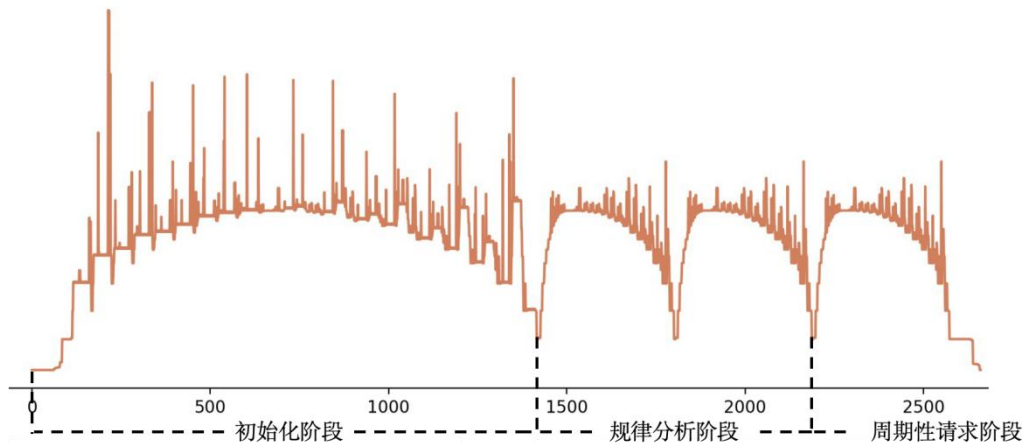


图 4.7 内存请求分区示意图

内存合并技术是指对相邻的空闲 $Chunk$ 进行合并, 如果 $Chunk_i$ 为空闲 $Chunk$, 且与该 $Chunk$ 使用双向链表进行关联的 $Chunk_{i-1}$ 或者 $Chunk_{i+1}$ 也是空闲 $Chunk$, 则合并相邻的空闲 $Chunk$ 。以合并 $\{Chunk_i, Chunk_{i+1}\}$ 为例, 合并之后, 按照式 (4.2) 更新 $Chunk_i$ 的尺寸, 在更新完成后, 删除 $Chunk_{i+1}$ 。

$$Chunk_i.size = Chunk_i.size + Chunk_{i+1}.size \quad (4.2)$$

内存整理技术则对数据在空间中的布局进行彻底整理, 如图 4.8 所示, 从前往后遍历 $Chunk$ 链表, 当已分配的 $Chunk_i$ 前面存在空闲 $Chunk_{i-1}$, 首先, 标记 $Chunk_i$ 为已使用, 其次, 则将 $Chunk_i$ 中数据复制到 $Chunk_{i-1}$ 的起始地址, 互换 $Chunk_{i-1}$ 和 $Chunk_i$ 的尺寸, 标记 $Chunk_i$ 为空闲状态。当存在两个相邻 $Chunk$ 均为空闲状态时, 则进行内存合并, 通过这样的方式将分布在已分配内存中的内存碎片收集到一起。

在初始化及规律分析阶段, 采用内存整理技术以最小化内存占用。首先, 在处理数据分配请求时, 先设备内存行长度对齐请求数据长度的策略。其次, 该方法引入了一个参数 $small_request_size$, 用于判别内存请求的分配区域。对于尺寸小于 $small_request_size$ 的内存请求, 在 $Block1$ 中的小请求数据区 M_{small} 中通过 $Chunk$ 进行管理。对于尺寸大于 $small_request_size$ 的数据, 则交由 $Block2$ 中的常规请求数据区 M_{reg} 来进行内存的分配和释放。接着, 在常规请求数据区 M_{reg} 中进行内存分配时, 方法首先检查是否存在大于请求数据

尺寸的空闲 Chunk。若存在，则选择与请求尺寸差异最小的的 Chunk 进行分配。若未找到合适的空闲 Chunk，方法将对当前区域的 Chunk 链表进行内存整理。方法从前往后遍历 M_{reg} 区域中的 Chunk 链表进行内存整理，直到整理出的空闲 Chunk 尺寸可以满足该请求尺寸。在返回分配结果之前，会对寻找到的空闲 Chunk 进行切分，按照请求尺寸切分为 $\{Chunk_i, Chunk_{i+1}\}$ ， $Chunk_i$ 为对齐后的请求尺寸， $Chunk_{i+1}$ 为当前寻找的空闲 Chunk 尺寸与请求尺寸之差，标记 $Chunk_i$ 为使用状态后返回分配结果。最后，如果当通过内存整理方法仍然无法找到满足请求的 Chunk 时，尝试在 Block2 中申请新的 Chunk 来满足该内存请求，当 Block2 中剩余内存仍然无法满足该请求时，此次分配失败。特别的，请求分配过程和 CUDA Stream 绑定，确保数据移动和数据计算能够按序进行。同时针对数据拷贝引起的地址变化问题，提供了一种通过虚拟地址与真实地址转换的管理机制。在方法中，对请求者返回的是虚拟地址，而当需要进行实际数据访问时，则提供对应 Chunk 中的真实内存起始地址。

分配请求：6MB

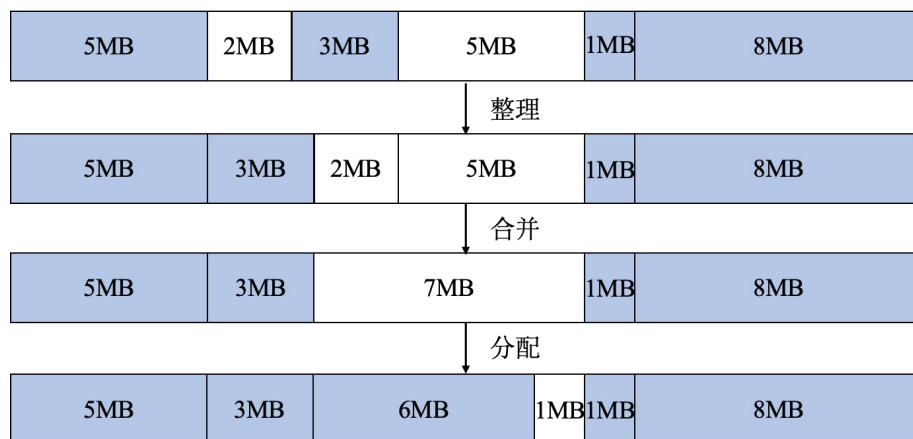


图 4.8 内存整理流程图

在周期性请求阶段，通过内存合并技术实现内存的高效复用。首先，根据内存行的长度对齐请求数据。其次，将尺寸小于 $small_request_size$ 的请求交给小请求数据区 M_{small} 进行管理，大于该尺寸的数据由 Block2 中的常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} 共同管理，在进行内存分配过程中，首先对 M_{reg} 区域和 M_{tmp} 的所有空闲 Chunk 进行合并，寻找合并后的 Chunk 中是否存在可以满足请求尺寸 Chunk，如果存在，对该 Chunk 进行切分并返回，如果不存在，则尝试在 Block2 中申请新的 Chunk 来满足该内存请求，当 Block2 中剩余内存仍然无法满足该请求时，此次分配失败。

其中，在周期性请求阶段的开始时，MARSP 将对 M_{reg} 区域中的所有 Chunk 进行内存整理，目的是为了确在训练过程中频繁使用的参数和缓存数据能够被紧凑地存储在低地址空间，而非分散于内存空间中。尽管使用内存整

理方法可以最大程度利用设备中的内存空间，然而此过程中频繁的数据拷贝操作会影响模型的训练速度。所以本方法仅在初始化阶段和规律分析阶段使用内存整理的方法优化内存空间使用情况，对于周期性请求阶段的内存请求则使用内存合并技术来进行管理。

4.5.2 双端数据分配方法

在模型训练过程中，周期内请求特征分析算法根据数据访问的特性，将数据分为临时数据和非临时数据两类。特别是对于周期性请求中的临时数据，由于其占用空间大且申请与释放的时间间隔短，这类数据容易在内存中产生大量碎片。为了有效管理周期性请求阶段的数据，采用了双端数据分配策略，旨在最大化保留连续的内存空间。

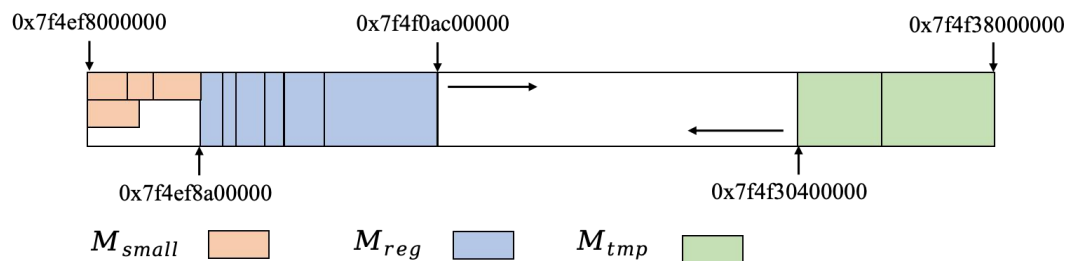


图 4.9 双端分配示意图

当进入周期性请求阶段时，方法管理的内存空间中，最前端是预留给固定尺寸小请求数据的区域 M_{small} ，随后是常规请求数据区 M_{reg} ，其中包含了多个排布紧密，且位于 Block2 头部区域的 Chunk 块，主要存储了模型训练过程中的参数信息。在周期性请求阶段，本方法将周期性请求中的非临时数据请求分配给常规请求数据区 M_{reg} ，在常规请求数据区 M_{reg} 中没有空闲 Chunk 的情况下，逐渐向 Block2 的尾部，即 Block2 中的高地址区域申请新的 Chunk 块；对于周期性请求中的临时数据请求，则分配给临时请求数据区 M_{tmp} ，该区位于 Block2 末端的高地址区域，采用从后往前划分 Chunk 块的方法来分配内存，并且当 M_{tmp} 中低地址的 Chunk 块在使用完毕后，该部分内存占用会被直接释放。

常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} 这两块区域的尺寸都是不固定的，其中， M_{reg} 的大小向高地址浮动， M_{tmp} 的尺寸向低地址浮动，而 M_{reg} 和 M_{tmp} 之间可以保留一片连续的未使用内存，这为多任务环境下的内存分配提供了操作空间。

4.6 本章小结

本章详细介绍了基于 AI 训练任务访存特征的内存分配算法，MARSP，该方法旨在优化模型训练场景下的内存请求分配流程。MARSP 采用内存请求规律分析算法识别请求数据中的周期规律及周期内的数据特征，同时基于多级分区的内存数据管理设备内存空间。根据分区管理的数据结构和标识特征的请求信息，采用分时双端内存分配管理方法来为数据请求序列分配内存空间。方法通过对内存空间的紧密排布和高效复用，减少训练过程中产生的内存碎片，从而实现对设备内存资源的高效管理。

第五章 实验

在本文的第三章和第四章分别介绍了基于双智能体强化学习的自适应内存优化方法（REMEM）和基于 AI 训练任务访存特征的内存分配算法（MARSP），用于对模型训练过程中内存资源进行管理。在本章内，将对本研究提出的 REMEM 方法和 MARSP 方法的有效性进行分析。针对 REMEM 方法，本文测试了 REMEM 方法对模型训练 batch size，训练吞吐量的提升效果，并比较 REMEM 方法中的双智能体搜索算法的有效性。针对 MARSP 方法，实验测试了在多个模型的训练访存请求序列下，训练过程中的最大内存需求量和内存碎片情况，并且对 MARSP 方法中的分时数据管理方法和双端数据分配方法的有效性进行分析。

5.1 REMEM 有效性实验分析

本研究在 TensorFlow 框架的基础上对 REMEM 方法进行了实现，并对比 TensorFlow 原生框架，TensorFlow 中的自适应内存优化方法，以及 Capuchin 和 HOME 这两个自适应内存优化方法，以验证 REMEM 方法对于模型训练过程中的内存优化效果。分别从模型最大训练 batch size、模型训练吞吐量、双智能搜索算法对吞吐量的影响等多个方面进行训练和评估。

5.1.1 实验设置

（1）测试模型介绍：

在本研究中，采用了五种广泛使用的深度学习模型进行训练，以评估不同内存管理策略的有效性。这些模型分别是 VGG16^[64]、MobileNet^[68]、ResNet-50^[69]、InceptionV3^[70]和 DenseNet-121^[71]。每个模型都代表了深度学习领域的重要进展，涵盖了从经典架构到最新创新的广泛范围。

VGG16，由牛津大学的视觉几何组（Visual Geometry Group）提出，是一种深层卷积神经网络，以其简洁的架构著称。它通过重复使用小尺寸的卷积核和最大池化层来逐步增加网络深度，有效地捕获图像的层次特征。VGG16 特别适合于图像识别和分类任务，其结构的深度和广泛的应用证明了深层网络在视觉任务中的有效性。

MobileNet, 由 Google 提出, 旨在优化移动和嵌入式设备上的性能, 通过深度可分离卷积来大幅减少计算量和模型大小, 同时保持较高的准确率。这种模型特别适合于在计算能力受限的环境中进行实时图像处理和分类任务。

ResNet-50 是残差网络 (Residual Network) 的一种, 由微软研究院提出。它引入了残差学习的概念来解决深度网络训练中的梯度消失问题, 通过引入跳跃连接允许信号直接传递。ResNet-50 通过这种创新架构显著提高了训练深度神经网络的效率和准确性, 成为深度学习领域的一个里程碑。

InceptionV3, 是 Inception 模型的第三个版本, 由 Google 研发。该模型通过采用多尺度卷积核和引入辅助分类器来改进性能和减少过拟合, 其特点是在保持计算效率的同时提高了网络的识别能力。

DenseNet-121, 是密集连接卷积网络 (Densely Connected Convolutional Networks) 的一种, 通过特征重用最大化了网络的效率。在 DenseNet 中, 每一层都直接与前面的所有层相连, 从而实现了改进的信息流和减少了参数的数量。

这些模型不仅各自在深度学习领域内具有重要的地位, 而且它们的结构多样性也代表了当前深度学习技术的不同方向和挑战。通过选择这五个模型进行训练测试, 有助于全面评估 REMEM 等自适应内存优化方法在不同类型的深度学习任务和模型架构中的效果。

(2) 设备和软件:

本论文实验硬件环境为单台服务器, 包含内存大小为 32GB 的 Intel(R) Xeon(R) CPU 和内存大小为 12GB 的 NVIDIA Tesla P100 高性能 GPU 计算设备, CPU 和 GPU 间的总线接口类型为 PCIe3.0*16。

(3) 软件环境配置:

软件环境配置如表 5.1 所示:

表 5.1 实验采用软件环境配置情况

系统版本	Ubuntu 18.04
Linux 内核版本	Linux 4.15.0-213
显卡驱动版本	NVIDIA-515.65.01
CUDA 版本	CUDA 11.7
Python 版本	Python 3.8.13
TensorFlow 版本	TensorFlow2.10.0

5.1.2 模型训练 batch size 分析

在深度学习模型的训练过程中, 训练样本的 batch size 对内存消耗有显著

影响。增加样本输入的 batch size 会放大模型训练过程中的特征数据和梯度数据的尺寸，从而增大训练过程中的内存需求，提高模型训练效率。本小节在同一设备上分别使用 TensorFlow 原始训练方法，TensorFlow 中的自适应内存优化方法以及 Capuchin 方法、HOME 方法以及 REMEM 方法进行模型训练，并通过测试不同方法可进行训练的最大 batch size 来验证自适应内存优化方法的有效性。其中，REMEM 方法迭代 500 轮进行策略搜索，并设置时间间隔搜索系数 k 为 0.9 以平衡策略搜索时间和策略搜索效果。

如表 5.2 所示，本实验分别在 VGG16、MobileNet、ResNet-50、InceptionV3 以及 DenseNet-121 这五个常用模型上进行方法测试。其中 InceptionV3 的输入尺寸设置为 $[N, 3, 75, 75]$ ，其他模型的输入尺寸均设置为 $[N, 3, 32, 32]$ 。

表 5.2 不同方法下模型最大训练 batch size

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
TensorFlow	8968	8274	4898	2024	2520
TF 优化	9423	8183	5202	2100	2795
Capuchin	9512	8261	5252	2120	2822
HOME	9698	8423	5355	2162	2849
REMEM	10250	8435	5359	2167	2915

观察表 5.2 中的数据可知，采用自适应内存优化方法对模型进行训练，相较于 TensorFlow 的标准训练方法，能显著增加模型训练过程中最大 batch size。这归功于自适应内存优化方法中对于重计算技术和内存交换技术的应用，该方法通过卸载训练加速器内存中正向传播过程所产生的部分中间特征数据，有效减少了在内存瓶颈阶段（即正向传播结束，反向传播开始阶段）的内存占用。这一机制使得模型能够处理更大 batch size 的样本输入，从而提高训练效率。

进一步的，通过在五种常用的深度学习模型上进行的实验验证，本研究发现，与其他自适应内存优化策略相比，采用 REMEM 方法能够最大化模型训练 batch size。其中，在 VGG16 模型和 DenseNet-121 模型上的应用中，REMEM 相较于 TensorFlow 的原始训练方法，分别实现 14.2% 和 15.6% 的效果提升，与 HOME 自适应优化方法相比，则分别提升了 5.6% 和 2.3%。然而，值得注意的是，在部分模型如 ResNet-50 上，REMEM 方法相较于其他自适应内存优化方法的策略提升有限，这是因为其他自适应内存优化策略已经对大部分可优化的数据进行了有效处理。模型结构确定了训练过程中可优化的特征数据的数量，也决定了自适应内存优化策略的搜索空间。这一发现强调了在实现内存优化过程中，模型结构的限制对策略优化潜力的影响。

5.1.3 模型训练吞吐量分析

本小节对模型训练过程中的吞吐量进行测量分析。模型训练吞吐量是指在训练深度学习模型过程中系统每单位时间内能处理的数据量，通常以样本/秒 (samples/second) 为单位，表示每秒可以处理多少个样本，训练吞吐量是评估优化训练策略的重要指标。吞吐量越大，代表处理相同的测试样本所花费的训练时间越少，在模型训练过程中每个 step 所需要的时间更少。在下文中，将首先比较 TensorFlow 标准训练方法，TensorFlow 中的自适应内存优化方法以及 Capuchin 方法、HOME 方法以及本文提出的 REMEM 方法使用最大训练 batch size 进行训练过程中吞吐量数据；然后，分析在 VGG16 模型的训练过程中时，使用不同训练方法并逐步放大训练 batch size 时对吞吐量的影响。其中，REMEM 进行对每个模型进行 500 轮次的策略搜索，并设置时间间隔搜索系数 k 为 0.9。

表 5.3 最大训练 batch size 下的模型训练吞吐量

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
TensorFlow	3628	4714	4301	2104	2299
TF 优化	3214	4186	3950	2075	2299
Capuchin	2983	3989	3622	1786	2158
HOME	3674	4090	3976	2034	2387
REMEM	3960	4684	4154	2081	2409

表 5.3 中展示了不同训练方法在 VGG16、MobileNet、ResNet-50、InceptionV3 以及 DenseNet-121 模型上最大训练 batch size 下的模型训练吞吐量数据。该数据由最大训练 batch size 除以该 batch size 下的平均每轮训练时间得出，表明在当前训练条件下，系统每单位时间内能处理的数据量。

研究发现，在标准 TensorFlow 训练方法下，模型展现了较高的训练吞吐量，这主要是因为虽然自适应内存优化技术，如重计算和内存交换，能够有效减少训练过程中的内存需求，但它们同时引入了额外的计算和数据传输开销，从而可能降低训练效率。然而，通过采用 REMEM 方法中的双智能体搜索方法同时对训练过程中数据执行的内存优化策略和策略执行时间进行搜索，能够在很大程度上消减内存优化技术引入的时间代价。具体实验结果表明，REMEM 方法在 VGG16 和 DenseNet-121 模型上，相比于 TensorFlow 标准训练方法，分别实现了 9.1% 和 4.7% 的训练吞吐量提升。而在 MobileNet，ResNet-50 和 InceptionV3 模型上的应用中，REMEM 方法能够在保持相近的训练吞吐量的同时，训练更大 batch size 的样本数据，这表明 REMEM 方法能够在不牺牲训练

效率的前提下，提升训练时的内存使用效率。同时，与 Capuchin、HOME 等自适应内存优化方法相比较，REMEM 方法分别实现了最高 32.7%和 9.18%训练吞吐量提升，这是由于 REMEM 方法中通过对内存优化策略执行时间的搜索，减少了内存优化策略执行过程中的数据等待和数据同步过程，降低了重计算和内存交换方法对模型训练时间的影响，进一步证明了 REMEM 在内存优化和训练效率平衡方面的优势。

在本研究中，采用不同 batch size 的样本输入对 VGG16 模型进行训练，收集并统计了在各个特定 batch size 下，不同训练策略下模型的训练吞吐量，并据此绘制了图 5.1。该图展示了在 batch size 范围 8500 至 10100 之间，采用 TensorFlow 的标准训练方法、TensorFlow 内置的自适应内存优化方法、Capuchin 自适应内存优化方法、HOME 自适应内存优化方法以及 REMEM 方法对 VGG16 模型进行训练时，训练吞吐量的动态变化情况。

图 5.1 的分析结果揭示，在整个模型训练过程中，REMEM 方法展现了较为稳定的吞吐量波动，并且在多个训练 batch size 中，相比其他自适应内存优化方法，REMEM 方法能够实现最高的训练吞吐量。相较于 TensorFlow 的标准训练方法，HOME 方法在多个 batch size 中也实现了吞吐量的提升，但其提升效果相对于 REMEM 方法来说较为有限。这种差异主要源于 REMEM 方法在优化过程中，通过对策略执行时间间隔的精确搜索，有效地减少了内存优化导致的重复计算以及数据等待，从而在扩大模型训练 batch size 的同时，最大程度减少了对训练时间的延长。

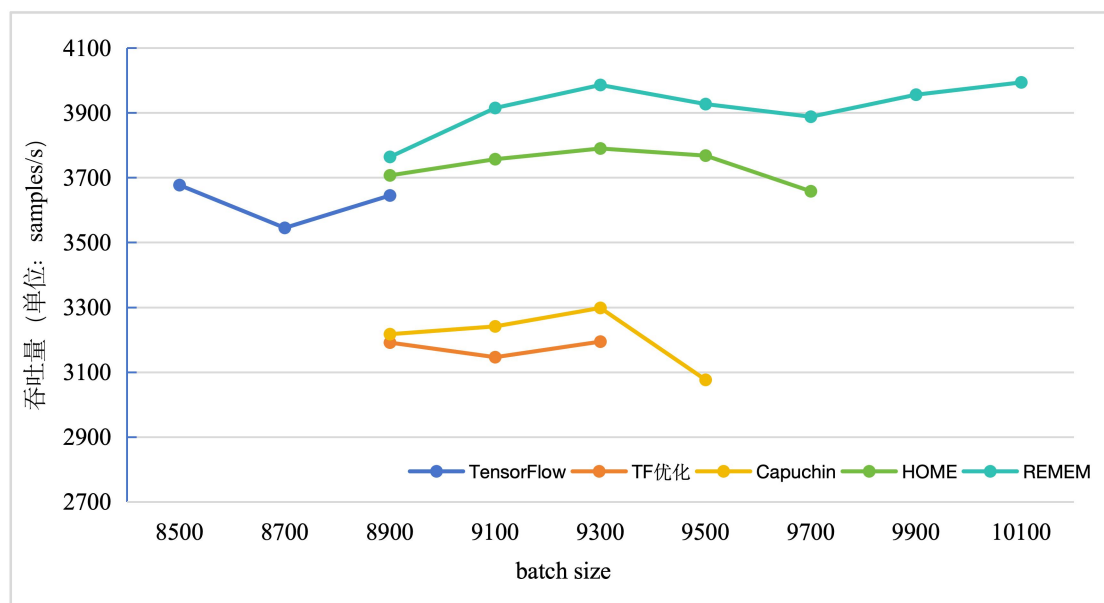


图 5.1 不同 batch size 下 VGG16 模型的训练吞吐量变化

5.1.4 双智能体搜索算法有效性分析

为了评估 REMEM 方法中双智能体搜索算法的有效性, 本研究设计实验对比单智能体与双智能体的优化策略搜索结果的应用性能差异。具体而言, 实验首先采用单智能体进行优化策略搜索, 其中时间间隔搜索系数 k 设定为 1, 以此设置下, 策略的执行时间间隔被限定为唯一选择, 即数据两次使用间的最大时间间隔。随后, 实验通过双智能体同时对优化策略及其执行时间进行搜索, 以探索和验证双智能体方法在优化效果上的潜在优势。

在本实验中, 两种算法基于 VGG16、MobileNet、ResNet-50、InceptionV3 和 DenseNet-121 模型信息进行 500 轮迭代搜索, 并将所得到的策略序列应用于对应模型的训练过程中, 以比较这两种方法在支持的最大模型训练 batch size 以及相应 batch size 下的模型训练吞吐量的差异。

表 5.4 不同搜索算法下的最大模型训练 batch size

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
REMEM 单智能体	9708	8431	5360	2164	2818
REMEM 双智能体	10250	8435	5359	2167	2915

对比分析表 5.4 中数据可以观察到, 在 VGG16 和 DenseNet-121 模型上, 双智能体搜索方法相较于单智能体搜索方法, 能够支持更高的训练 batch size。这表明通过同时优化策略及其执行时间, 双智能体搜索方法能够更有效地利用内存资源, 从而支持更大的训练 batch size, 进而可能提升训练效率和吞吐量。对于 MobileNet、ResNet-50 和 InceptionV3 模型, 尽管两种搜索方法下的最大训练 batch size 相近, 但双智能体搜索在细微调整和精细化优化策略执行时间方面仍然显示出其潜在的优势。如表 5.5 中所示, REMEM 双智能体搜索策略在所有测试模型上均实现了显著高于单智能体搜索策略的训练吞吐量。具体来说, 在 VGG16 模型上, 双智能体搜索策略的吞吐量为 3960, 相比单智能体搜索策略的 3553, 提升了约 11.5%。对于 MobileNet 模型, 双智能体策略实现了 4684 的吞吐量, 较单智能体的 3992, 提升了约 17.3%。对于 ResNet-50, InceptionV3 和 DenseNet-121 模型上, 则分别提升了 11.1%, 3.1%和 20.2%。

表 5.5 最大训练 batch size 下的模型训练吞吐量

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
REMEM 单智能体	3553	3992	3740	2019	2005
REMEM 双智能体	3960	4684	4154	2081	2409

实验结果表明, 双智能体搜索算法在调整优化策略及其执行时间上的综合优势, 与只搜索优化策略的单智能体搜索相比, 双智能体方法可能更能有效地

平衡训练效率和内存使用，从而在大规模数据训练场景中实现更优的训练 batch size 和吞吐量。

5.2 MARSP 有效性实验分析

本节将通过对比实验评估 MARSP 方法的有效性，分别比对相同设备下，模型训练最大内存需求量、内存碎片情况、分时数据管理方法对训练最大内存需求量的影响、双端数据分配方法对内存碎片的影响等几个方面进行性能评估实验及分析。为了验证 MARSP 策略的有效性，本研究将对比常用深度学习训练框架 PyTorch 以及 TensorFlow 中的内存分配算法。

5.2.1 实验设置

(1) 测试数据介绍:

为了验证 MARSP 的有效性，本研究收集了 VGG16、MobileNet、ResNet-50、InceptionV3 和 DenseNet-121 这五个常用深度学习模型在实际训练过程中的内存分配和释放请求，并对这些请求进行格式整理和数据绑定，以模拟真实训练过程中的数据请求序列。

具体来说，本研究修改并重新编译 TensorFlow 框架的源码，在其数据分配流程的前后添加数据收集器，并使用修改后的 TensorFlow 框架训练 VGG16、MobileNet 等模型，其中五个模型的数据输入尺寸均设置为[32,3,224,224]。在模型训练过程中，TensorFlow 框架中的数据收集器收集内存请求信息并输出到文件。表 5.6 展示了内存分配请求跟踪文件的部分样例：

表 5.6 内存分配请求跟踪文件样例

请求操作	设备名称	请求内存尺寸	地址
MemoryAllocation	gpu_host_bfc	22861824	0x7f0ada000000
MemoryAllocation	GPU_0_bfc	58982400	0x7f0bfce6d700
MemoryAllocation	GPU_0_bfc	9437184	0x7f0c04d01000
MemoryAllocation	GPU_0_bfc	339738624	0x7f0c05b0e700
MemoryDeallocation	GPU_0_bfc	339738624	0x7f0c05b0e700
MemoryDeallocation	GPU_0_bfc	9437184	0x7f0c04d01000

其中，请求操作标识了该内存请求的操作内容，MemoryAllocation 代表该请求为内存分配请求，MemoryDeallocation 代表该请求为内存释放请求。设备名称代表该此请求操作的位置，gpu_host_bfc 表示该请求作用于 host 端，即

CPU 的内存空间, GPU_0_bfc 说明该请求作用于 device 端, 即训练加速设备 (GPU) 的内存空间, 请求内存尺寸则代表请求数据的大小, 地址代表给该分配请求分配的数据地址, 或者根据该地址对数据占用的空间执行释放操作。

本研究对收集的内存请求信息进行整理, 首先过滤掉 host 端的内存请求信息, 只保留对于 GPU 内存的请求信息, 并按照请求顺序分配编号; 其次根据内存请求的地址将内存分配和内存释放请求进行绑定, 每个内存释放请求向前绑定最近一个相同地址的分配请求, 而内存分配请求对应的分配请求编号为-1。按照此规则形成一个 (请求尺寸, 分配请求标号) 的二元组序列。以表 5.6 为例, 表中的内存请求信息进行整理后会生成二元组序列[(58982400, -1), (9437184, -1), (339738624, -1), (339738624, 3), (9437184, 2)]。该序列还原了模型训练过程中的真实内存请求访问情况, 本文收集整理了多轮训练过程中, 五个常用深度学习模型的请求序列以测试 MARSP 方法的有效性。

本研究抽取了 TensorFlow 框架中的 BFCAllocator 的相关实现代码, 并根据 PyTorch 框架中 CudaCachingAllocator 的细节还原了 PyTorch 框架中自定义内存分配器。同时, 对于 PagedAttention 中的按页分配的思想和对本研究提出的 MARSP 方法进行了代码实现。在实验过程中, 将收集整理的训练内存请求序列输入到不同的内存分配管理方法中, 统计不同方法所需的训练资源消耗。

(2) 设备和软件:

本论文实验硬件环境为单台服务器, 包含内存大小为 32GB 的 Intel(R) Xeon(R) CPU 和内存大小为 12GB 的 NVIDIA Tesla P100 高性能 GPU 计算设备。

(3) 软件环境配置:

软件环境配置如表 5.7 所示:

表 5.7 实验采用软件环境配置情况

系统版本	Ubuntu 18.04
Linux 内核版本	Linux 4.15.0-213
显卡驱动版本	NVIDIA-515.65.01
CUDA 版本	CUDA 11.7
PyTorch 版本	PyTorch 2.2.0
TensorFlow 版本	TensorFlow 2.10.0
Gcc 版本	Gcc 9.4.0

5.2.2 训练最大内存需求量分析

在本小节中, 将对模型训练过程中的最大内存需求量进行分析。本研究使

用 PyTorch, TensorFlow 中的分配算法, PagedAttention 中的分配算法和 MARSP 方法在 VGG16、MobileNet、ResNet-50、InceptionV3 和 DenseNet-121 这五个模型的内存请求序列上进行效果验证。其中, PyTorch 中的 CudaCachingAllocator 在无法寻找到合适的内存空间时, 会向系统释放并重新申请内存空间, 本实验累加模型训练过程中 PyTorch 中的分配算法向系统申请和释放内存空间大小, 并取其中的最大值作为模型训练过程中的最大内存需求量, 并且考虑到测试设备的内存容量限制, 本实验设置了一块虚拟空间供 PyTorch 向设备申请内存。PagedAttention 中的内存分配算法会将管理的物理内存空间切分为多个固定尺寸的页面, 本实验将内存空间划分多个 32KB 的页面, 并统计训练过程中使用的页面数量, 取其中的最大值作为训练过程中的最大内存需求量。TensorFlow 中的 BFCAllocator 和 MARSP 方法都采取了预先向系统申请一整块内存作为内存池, 再使用分配算法对内存池进行管理的策略。因此, 对于 TensorFlow 中的分配算法和 MARSP 方法, 本研究测试不同的内存池尺寸, 并选择能够满足模型训练内存请求序列的最小预分配内存池尺寸作为模型训练过程中最大内存需求量。

在该实验中, 设置 MARSP 方法中小请求尺寸 $small_request_size$ 为 2MB, 小于该尺寸的请求交由小请求数据区 M_{small} 管理, 其中 M_{small} 尺寸固定分配为 10MB。临时分配请求的判别间隔 tmp_slot 设置为 2。

表 5.8 不同模型训练过程中的最大内存需求量 (单位: GB)

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
PyTorch	8.6425	7.1054	14.8867	11.2929	13.7460
TensorFlow	5.3764	4.8576	10.9687	5.4462	9.2128
PagedAttention	5.1603	4.7376	8.4703	4.5671	7.7357
MARSP	5.1660	4.7256	8.4451	4.5330	8.4421

由表 5.8 可知, 根据实验的数据, 可以观察到 MARSP 在多个测试的模型上都实现了更低的内存消耗。其中 PyTorch 相较于 TensorFlow, PagedAttention 和 MARSP 明显展现出更多的内存需求, 这是因为 PyTorch 的内存分配算法为了兼容多任务训练过程, 没有选择预分配内存的方案, 而是采取动态向设备申请扩容的策略。这也导致 PyTorch 中分配算法无法高效复用内存, 并且多次向设备申请的内存空间通常不连续, 导致内存中存在大量外部碎片, 这些因素都导致 PyTorch 中的分配算法对内存的需求偏高。

图 5.2 展示了 PyTorch 和 TensorFlow 中及 PagedAttention 中的内存分配方法所需的训练内存容量, 相对 MARSP 方法的训练所需内存容量的比例。观察可知, MARSP 方法在多个测试模型请求序列上均实现更少的内存消耗。具体来

说, 相较于 MARSP 方法, TensorFlow 和 PyTorch 在 VGG16 模型上的内存消耗分别增加 4.1%和 67.3%, 在 MobileNet 模型上增加了 2.8%和 50.4%, 在 ResNet-50 模型上增加了 22.9%和 73.6%, 在 InceptionV3 模型的训练内存请求序列上, TensorFlow 和 PyTorch 中内存消耗的峰值分别是 MARSP 方法的 1.201 倍和 2.491 倍, 在 DenseNet-121 模型上, 则分别是 1.091 倍和 1.628 倍。

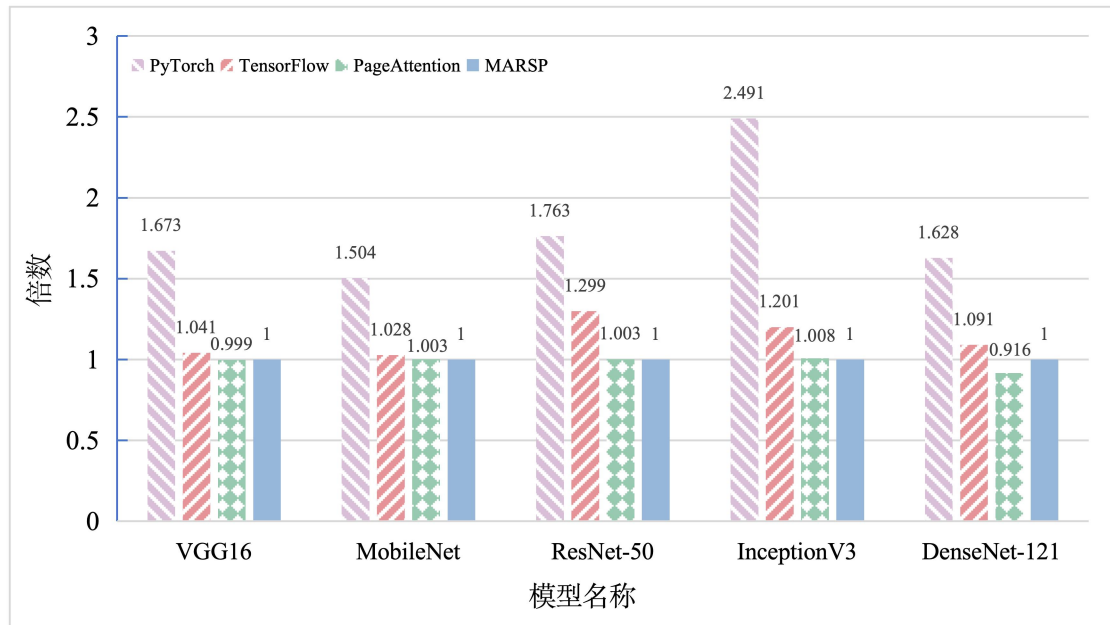


图 5.2 不同模型内存请求序列下的内存消耗情况

MARSP 方法和 PagedAttention 中的分页管理方法相比, 两者训练模型时所需的内存容量相近, 这是因为 PagedAttention 方法中使用分页方法管理内存空间, 完全消除了分配过程中的外部碎片, 所以在 VGG16 和 DenseNet-121 模型上, 使用分页内存管理方法所需的内存更少, 然而, 在进行实验测试过程中, 观察到在所有的测试模型请求序列上, PagedAttention 方法中的分页内存管理方法均需要更多的时间来进行内存的分配和释放。如在处理 DenseNet-121 模型训练内存请求序列时, 分页方法管理需要 46.5216s 而 MARSP 方法仅需要 0.8913s。这些结果表明, MARSP 提供了一种更为高效的内存管理机制。

MARSP 方法的有效性主要归因于其针对模型训练过程中的内存请求特征进行的优化的策略。包括使用多级分区内存结构, 对于内存请求规律的动态分析以及分时数据管理方法和双端数据分配方法。这些技术能够明显减少在深度学习模型训练过程中的内存需求, 从而允许在有限的硬件资源上训练更大或更复杂的模型。

5.2.3 内存碎片分析

在内存分配过程中，内存碎片是造成模型训练过程内存浪费的主要原因之一。在本小节将比较 MARSP 方法同 PyTorch，TensorFlow 以及 PagedAttention 中的内存分配算法在训练相同模型时产生的内存碎片大小差异。本实验测量了 VGG16、MobileNet、ResNet-50、InceptionV3 以及 DenseNet-121 模型的内存请求序列下的内存碎片尺寸并对其进行分析。具体而言，实验收集了内存请求和分配算法实际分配的内存空间大小。在实际分配的内存空间中，只有请求大小的空间会被使用，剩余的部分则形成了无法使用的内部碎片。该实验统计了多个模型内存请求序列下不同内存分配算法产生的内部内存碎片尺寸。

MARSP 方法的实验设置：小请求尺寸 $small_request_size$ 为 2MB， M_{small} 尺寸固定设置为 10MB。临时分配请求的判别间隔 tmp_slot 的值设置为 2。

表 5.9 不同模型内存请求序列下的内存碎片（单位：B）

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
PyTorch	7864880	8059691	46869923	34555771	48176787
TensorFlow	269566640	276016755	343224575	251687623	444874387
PagedAttention	2401068	40199263	72742283	30490855	16298139
MARSP	3496	17063	55399	77987	134991

表 5.9 中展示了不同模型内存请求序列下的内存碎片情况，观察数据可知，TensorFlow 中分配方法所产生的内存碎片的尺寸普遍大于 PyTorch 和 MARSP 和 PagedAttention 中的分配方法所产生的内存碎片。主要原因是 TensorFlow 中的请求按照固定值 256B 进行对齐，按照对齐后的请求尺寸寻找合适的 Chunk 块。而在 MARSP 方法中，采用了按缓存行对齐的策略，并且在数据分配 Chunk 时，会寻找尺寸最接近的低地址 Chunk，同时在分时数据管理方法中，会将散落的内存碎片进行整理后再次分配。这些策略显著减少了模型训练过程中分配所产生的内存碎片尺寸，从而使得在多个模型内存请求序列的测试结果上，MARSP 方法均保持了最少内存碎片的成果。

5.2.4 分时数据管理方法有效性分析

为了验证 MARSP 方法中分时数据管理方法的有效性，本小节设置了实验对比常规请求数据区 M_{reg} 中的两种 Chunk 管理方案对训练过程中的最大内存

需求量的影响。分别是使用内存合并对 M_{reg} 区域进行管理，以及根据请求的周期特性分阶段组合内存整理和内存合并技术进行管理。本实验针对这两种方案，分别在 VGG16、MobileNet、ResNet-50、InceptionV3 和 DenseNet-121 模型训练请求序列上进行了效果验证。实验中，MARSP 对小请求尺寸判别尺寸 $small_request_size$ 设置为 2MB， M_{small} 区域大小固定设置为 10MB。临时分配请求的判别间隔 tmp_slot 的值设置为 2。

表 5.10 不同数据管理方案下的训练最大内存需求量 (单位: GB)

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
内存合并	5.1660	4.7683	9.0276	5.4867	9.4279
分时数据管理	5.1660	4.7256	8.4451	4.5330	8.4421

首先，根据表 5.10 中的数据可以观察内存合并技术对训练最大内存需求量的影响。内存合并作为一种优化手段，通过合并多个小的空闲内存块为较大的内存块，旨在减少内存碎片化，能够有效地管理内存资源，使用内存合并技术对于内存需求较大的模型训练请求序列如 ResNet-50，模型训练过程中的内存消耗峰值仅为 9.0276GB。

进一步地，分时数据管理方法的应用结果表明，该技术在所有测试模型上都实现了更低的训练最大内存需求量。例如，在 MobileNet 模型训练请求序列上，分时数据管理方法将内存需求量从仅应用内存合并技术的 4.7683GB 降低到了 4.7256GB；在 ResNet-50 上，该方法将内存需求量从 9.0276GB 降低至 8.4451GB。分时数据管理方法根据模型训练过程中的周期特征，对于初始化及规律分析阶段，采用内存整理技术紧密存储训练过程中的持久性数据；在周期性访问阶段，通过内存合并技术提高内存利用效率。表 5.10 的实验测试结果强调了分时数据管理技术同时利用了内存整理结合内存合并技术带来的优势。

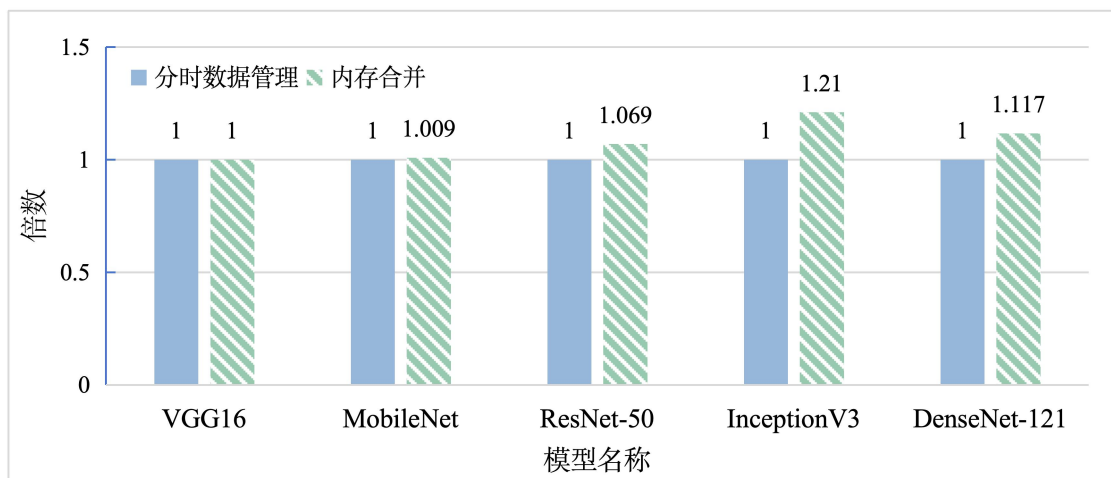


图 5.3 分时数据管理方法对最大内存需求量的影响

图 5.3 提供了一个直观的比较，展示了在不同模型内存请求序列中，分时

数据管理方法相较于内存合并技术对内存资源需求的相对比例。特别是在训练具有复杂结构的模型如 ResNet-50、InceptionV3 和 DenseNet-121 时，分时数据管理方法的优势尤为显著。相较于分时数据管理方法，采用内存合并技术处理这些模型的训练请求序列，分别需要额外增加 0.069 倍、0.21 倍和 0.117 倍的内存空间。对于这些具有大量参数数据的复杂模型，MARSP 中的分时数据管理方法通过内存整理技术重新组织和优化参数数据在内存中的布局，降低了内存碎片化，进而减少了模型训练过程中的内存需求。

5.2.5 双端数据分配方法有效性分析

本小节将考察不同的临时访问间隔 (tmp_slot) 下，周期性请求阶段能够保留的最大连续内存块的尺寸，并根据实验结果分析论文 4.5.2 节中提出的双端数据分配方法对内存碎片的影响。实验配置了 10GB 的初始内存空间，作为 MARSP 的预分配内存池，通过测量常规请求数据区 M_{reg} 和临时请求数据区 M_{tmp} 之间的空闲内存区域尺寸来对双端数据分配方法的有效性进行分析。特别地，当时临时分配请求判别间隔 tmp_slot 设置为 0 时，此时周期性请求阶段的所有数据请求均由常规请求数据区 M_{reg} 进行管理，而设置 $tmp_slot = \{1, 2, 3, 4\}$ ，则可调整临时请求数据区 M_{tmp} 和常规请求数据区 M_{reg} 处理的数据请求比例。实验同时设定小请求尺寸判别尺寸 $small_request_size$ 和 M_{small} 区域大小分别为 2MB 和 10MB。

表 5.11 双端数据分配方法的空闲内存区域大小 (单位: GB)

方案\测试模型	VGG16	MobileNet	ResNet-50	InceptionV3	DenseNet-121
$tmp_slot = 0$	3.5650	6.3431	5.0727	6.3142	2.0965
$tmp_slot = 1$	5.3822	7.6272	5.5874	6.6991	2.2401
$tmp_slot = 2$	5.5465	7.6274	5.5726	6.6766	2.2401
$tmp_slot = 3$	5.5525	7.2662	5.2823	6.6446	1.8385
$tmp_slot = 4$	5.8759	7.5194	5.2684	6.6358	1.8277

由表 5.11 中的数据可知，设置 tmp_slot 为不同值，临时请求数据区 M_{tmp} 和常规请求数据区 M_{reg} 管理的分配请求数量和区域的尺寸也随之浮动。双端数据分配方法将周期性请求中的临时数据分配到高地址的临时请求数据区 M_{tmp} ，将非临时数据分配到低地址的常规请求数据区 M_{reg} ，旨在保留两个数据区域之间的连续内存空间，以减少周期性请求阶段的内存需求。观察数据可知，由于模型结构的差异性，双端数据分配比例的最优设置在不同的模型结构上存在差异。例如，在 ResNet-50 和 InceptionV3 的请求序列上，设置 $tmp_slot = 1$ 时，

M_{tmp} 和 M_{reg} 之间保留的空闲内存尺寸最大；对于 MobileNet，则在 tmp_slot 设置为 2 时，区域间的空闲内存最大。此外，与仅使用常规请求数据区 M_{reg} 管理周期性请求数据（ $tmp_slot = 0$ ）的情况相比，采用双端数据分配方法的测试结果普遍表现出优势，这证明了双端数据分配算法在减少周期性请求阶段的内存需求方面的有效性。

5.3 本章小结

为了验证本文方法的有效性，本章在 TensorFlow 的基础上设计并实现了 REMEM 方法，本文针对深度学习常用模型 VGG16、MobileNet、ResNet-50、InceptionV3 和 DenseNet-121 进行模型最大训练 batch size 和模型训练吞吐量的测试。实验结果表明相较于 Capuchin、HOME 等自适应内存优化方法，REMEM 方法能在增加模型最大训练 batch size 的同时保持模型训练吞吐量，从而进行高效的内存优化。同时，本章实现了 MARSP 方法，并收集模型训练过程中的内存请求序列进行测试，分别在训练最大内存需求量，内存碎片，分时数据管理方法对训练最大内存需求量的影响以及双端数据分配方法对内存碎片的影响等几个方面进行性能评估实验及分析。本文提出的 MARSP 内存分配算法对比 PyTorch 和 TensorFlow 框架以及 PagedAttention 中的内存分配算法，显著减少了模型训练过程中内存消耗峰值和分配过程中产生的内存碎片，并且通过实验证明了 MARSP 方法中的分时数据管理方法和双端数据分配方法的有效性。

第六章 总结与展望

6.1 本文工作总结

随着深度学习的广泛应用，模型向更复杂、更智能的方向快速演进，这对训练加速设备的内存资源提出了更高的要求。尽管加速设备的内存资源在过去十年中显著增加，却远不及模型和数据集的增长规模，这导致内存成为限制模型训练和应用的关键因素。面对这一挑战，研究学者提出了多种内存优化方法以减少模型训练过程中内存需求，然而，由于模型结构的差异性，现有的自适应内存优化方法很难在节约内存空间的同时兼顾模型训练效率。同时，当前模型训练过程中的内存分配算法没有考虑到模型训练过程中数据访存特征，仍然面临因为内存碎片造成的资源浪费问题。因此本文针对上述问题，对现有的模型训练过程中的内存优化方法进行优化，主要工作如下：

(1) 针对模型结构差异导致自适应内存优化方法执行效率低下问题，本文提出基于双智能体强化学习的自适应内存优化方法。首先，该方法通过 Profiling 技术收集神经网络模型信息及训练设备信息，基于这些信息，构建训练状态矩阵，用以模拟神经网络训练过程中的内存资源、计算资源和通信资源的消耗状态；其次，采用两个强化学习智能体 `action_agent` 和 `time_agent` 来协同搜索最优内存优化策略和优化策略执行时间；然后，在状态矩阵中动态模拟策略搜索结果在实际环境中执行情况，并根据资源使用情况计算奖励值，从而评估不同策略的效益；最后，通过多轮迭代优化，以得到收益最大的策略搜索结果，根据策略搜索结果优化计算图中的数据依赖关系，避免资源利用冲突，在提升模型最大训练 batch size 的同时保持模型训练效率。经实验验证，使用 REMEM 方法搜索的策略对神经网络模型进行内存优化后，有效提高了设备上模型训练最大 batch size，同时相对于 Capuchin、HOME 等自适应内存优化方法，实现了最高 32.7%和 9.18%训练吞吐量提升。

(2) 针对模型迭代训练过程中产生的内存碎片问题，本文提出 MARSP，一种基于 AI 训练任务访存特征的内存分配算法。首先，MARSP 方法基于滑动窗口和距离差分方法对请求数据的访问规律进行分析，寻找数据请求访问周期的长度以及起始位置，结合模型训练特征区分周期内数据请求为临时数据请求和非临时数据请求；其次，引入了多级分区的内存管理方法，使用 Block 和 Chunk 两级数据管理结构对内存空间分区，最后，采用分时双端内存分配方法

来管理周期性数据请求，根据数据在模型训练过程中的特征将数据请求分配给不同的内存区域，以优化训练过程中数据在内存空间中的布局。实验结果表明，MARSP 方法有效减少了模型训练过程中产生的内存碎片和训练内存需求，同 MARSP 相比，Pytorch 中的内存分配算法需要多使用 50%-149% 的内存空间进行模型的训练过程，而 TensorFlow 方法在训练时则额外需要 2.8% 到 22.9% 的内存空间。

6.2 未来展望

本文主要在面向深度学习训练场景的内存管理领域，对训练过程中的内存消耗及训练效率进行研究，通过调查近年来国内外的相关领域研究，指出了在自适应内存优化方法，以及内存分配算法方面存在的一些问题，并针对性的提出了基于强化学习的自适应内存优化方法、以及基于请求访问特征的内存分配算法。但是，由于设备和模型结构的不断迭代，以及相关优化技术的不断发展，本论文的研究还存在一些局限性。接下来，本文将分析进一步的研究工作。

(1) 内存管理策略融合，本文提出的基于双智能体强化学习的自适应内存优化方法，与基于 AI 训练任务访存特征的内存分配算法，同时作用于模型训练环节，分别对模型训练过程中的数据分配与释放时机，及其对应的分配和释放请求进行管理与优化。在后续研究中，可以将以上两个策略进行融合并进一步研究。

(2) 网络模型扩展，本文测试所用的网络模型为图像领域的 VGG16、MobileNet、ResNet-50、InceptionV3 以及 DenseNet-121 网络，但随着模型结构的不断迭代，更多领域的深度学习模型和更大规模的网络模型可以被应用并验证本文工作的有效性，后续研究工作中，将对更多种结构和更大规模的网络模型进行分析和验证，并在自然语言处理、计算机视觉和数据挖掘等多个领域进行研究。

(3) 分布式环境拓展，本文主要针对模型在单机训练环境下的内存优化技术研究与应用。然而，目前分布式训练是进行大模型训练的热门技术，如何在多机情况下进行内存优化策略的搜索与应用，是进行后续研究的一个重要方向。

参考文献

- [1] Ofer D, Brandes N, Linial M. The language of proteins: NLP, machine learning & protein sequences [J]. Computational and Structural Biotechnology Journal, 2021, 19: 1750-1758.
- [2] Torfi A, Shirvani R A, Keneshloo Y, et al. Natural language processing advancements by deep learning: A survey [J]. arXiv preprint arXiv: abs/2003.01200, 2020.
- [3] Chen J, Chen J, Zhang D, et al. Using deep transfer learning for image-based plant disease identification [J]. Computers and Electronics in Agriculture, 2020, 173: 105393.
- [4] Sarvamangala D, Kulkarni R V. Convolutional neural networks in medical image understanding: a survey [J]. Evolutionary intelligence, 2022, 15(1): 1-22.
- [5] Chang Y, Wang X, Wang J, et al. A survey on evaluation of large language models [J]. ACM Transactions on Intelligent Systems and Technology, 2023.
- [6] 马玮良, 彭轩, 熊倩, 等. 深度学习中的内存管理问题研究综述 [J]. 大数据, 2020, 6(04): 56-68.
- [7] 冯杨洋, 汪庆, 谢旻晖等. 从 BERT 到 ChatGPT: 大模型训练中的存储挑战与技术发展 [J/OL]. 计算机研究与发展 :1-15[2024-03-20]. <http://kns.cnki.net/kcms/detail/11.1777.TP.20240108.1539.012.html>.
- [8] Li S, Zhao Y, Varma R, et al. Pytorch distributed: Experiences on accelerating data parallel training [J]. arXiv preprint arXiv:200615704, 2020.
- [9] Rasley J, Rajbhandari S, Ruwase O, He Y. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters [C]. proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, USA: ACM, 2020: 3505-3506.
- [10] Fan S, Rong Y, Meng C, et al. DAPPLE: A pipelined data parallel approach for training large models [C]. proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, USA: ACM, 2021:431-445.
- [11] 王恩东, 闫瑞栋, 郭振华, 赵雅倩. 分布式训练系统及其优化算法综述 [J]. 计算机学报, 2024, 47(01): 1-28.
- [12] Jiang A Q, Sablayrolles A, Roux A, et al. Mixtral of experts [J]. arXiv preprint arXiv:240104088, 2024.

- [13]Peddie J. The History of the GPU-Eras and Environment [M]. Germany: Springer Nature, 2023.
- [14]Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018.
- [15]Anil R, Dai A M, Firat O, et al. Palm 2 technical report [J]. arXiv preprint arXiv:230510403, 2023.
- [16]Pablo Villalobos A H. Trends in training dataset sizes [EB/OL]. [2022]. <https://epochai.org/blog/trends-in-trainingdataset-sizes>.
- [17]LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-based learning applied to document recognition [J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [18]Zhai X, Kolesnikov A, Houlsby N, Beyer L. Scaling vision transformers [C]. proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR), USA: IEEE, 2022: 12104-12113.
- [19]Wu H, Judd P, Zhang X, et al. Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation [J]. arXiv:abs/2004.09602, 2020.
- [20]Liu Z, Sun M, Zhou T, et al. Rethinking the value of network pruning [J]. arXiv preprint arXiv:181005270, 2018.
- [21]李运. 深度神经网络压缩与加速方法研究 [D]; 安徽: 中国科学技术大学, 2023.
- [22]Chen T, Xu B, Zhang C, Guestrin C. Training deep nets with sublinear memory cost [J]. arXiv preprint arXiv:160406174, 2016.
- [23]Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design [C]. proceedings of the 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), USA: IEEE, 2016: 1-13.
- [24]高赫然, 吴恒, 许源佳,等. 面向深度学习训练的内存交换机制综述 [J]. 软件学报, 2023, 34(12): 5862-5886.
- [25]Beaumont O, Eyraud-Dubois L, Shilova A. Efficient Combination of Rematerialization and Offloading for Training DNNs [C]. proceedings of the Neural Information Processing Systems, USA: MIT Press, 2021:23844-23857.
- [26]马阳. 深度学习系统内存重用及优化方法研究 [D]; 湖北: 华中科技大学, 2020.
- [27]Paszke A, Gross S, Massa F, et al. Pytorch: An imperative style, high-performance deep learning library [J]. Advances in neural information processing systems, 2019, 32: 1-12.

- [28] Abadi M, Agarwal A, Barham P, et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems [J]. arXiv:abs/1603.04467, 2016.
- [29] Shriram S B, Garg A, Kulkarni P. Dynamic Memory Management for GPU-Based Training of Deep Neural Networks [J]. 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2019: 200-209.
- [30] Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention [C]. proceedings of the 29th Symposium on Operating Systems Principles, USA: ACM, 2023: 611-626.
- [31] Bottou L, Curtis F E, Nocedal J. Optimization methods for large-scale machine learning [J]. SIAM review, 2018, 60(2): 223-311.
- [32] Ruder S. An overview of gradient descent optimization algorithms [J]. arXiv preprint arXiv:1609.04747, 2016.
- [33] Gholami A, Kim S, Dong Z, et al. A survey of quantization methods for efficient neural network inference [M]. Low-Power Computer Vision. UK: Chapman and Hall/CRC. 2022: 291-326.
- [34] Ren J, Rajbhandari S, Aminabadi R Y, et al. {ZeRO-Offload}: Democratizing {Billion-Scale} model training [C]. proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC 21), USA: MIT Press, 2021.
- [35] Rajbhandari S, Ruwase O, Rasley J, et al. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning [C]. proceedings of the Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, USA: ACM, 2021: 1-14.
- [36] Rhu M, O'Connor M, Chatterjee N, et al. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks [J]. 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA), 2017: 78-91.
- [37] Abdelfattah A, Tomov S, Dongarra J. Towards half-precision computation for complex matrices: A case study for mixed precision solvers on gpus [C]. proceedings of the 2019 IEEE/ACM 10th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA), USA: IEEE, 2019: 17-24.
- [38] Tang Y, Wang C, Zhang Y, et al. Delta: Dynamically optimizing gpu memory beyond tensor recomputation [J]. arXiv preprint arXiv:2203.15980, 2022.
- [39] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design | IEEE Conference

- Publication | IEEE Xplore [J]. 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2016: 1-13.
- [40]Mao J, Chen X, Nixon K W, et al. MoDNN: Local distributed mobile computing system for Deep Neural Network [J]. Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017: 1396-401.
- [41]Hildebrand M, Khan J A, Trika S N, et al. AutoTM: Automatic Tensor Movement in Heterogeneous Memory Systems using Integer Linear Programming [C]. proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, USA: ACM, 2020: 875-890.
- [42]Huang C-c, Jin G, Li J. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping [C]. proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, USA: ACM, 2020: 1341-1355.
- [43]Chen P, He S, Zhang X, et al. CSWAP: A Self-Tuning Compression Framework for Accelerating Tensor Swapping in GPUs | IEEE Conference Publication | IEEE Xplore [C]. 2021 IEEE International Conference on Cluster Computing (CLUSTER), USA:IEEE, 2021: 271-282.
- [44]Wang L, Ye J, Zhao Y, et al. Superneurons: dynamic GPU memory management for training deep neural networks [C]. proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, USA: ACM, 2018: 41-53.
- [45]Peng X, Shi X, Dai H, et al. Capuchin: Tensor-based gpu memory management for deep learning [C]. proceedings of the Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, USA: ACM, 2020: 891-905.
- [46]Patil S G, Jain P, Dutta P, et al. Poet: Training neural networks on tiny devices with integrated rematerialization and paging [C]. proceedings of the International Conference on Machine Learning, USA: ACM, 2022: 17573-17583.
- [47]Kondili E, Pantelides C C, Sargent R W. A general algorithm for short-term scheduling of batch operations—I. MILP formulation [J]. Computers & Chemical Engineering, 1993, 17(2): 211-27.

- [48]He S, Chen P, Chen S, et al. HOME: A Holistic GPU Memory Management Framework for Deep Learning | IEEE Journals & Magazine | IEEE Xplore [J]. IEEE Transactions on Computers, 2022, 72(3): 826-838.
- [49]Kennedy J, Eberhart R. Particle swarm optimization [C]. proceedings of the Proceedings of ICNN'95-international conference on neural networks, USA: IEEE, 1995: 1942-1948.
- [50]Chen P, He S, Zhang X, et al. Accelerating Tensor Swapping in GPUs With Self-Tuning Compression | IEEE Journals & Magazine | IEEE Xplore [J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(12): 4484-4498.
- [51]Nguyen D T, Je H, Nguyen T N, et al. ShortcutFusion: From tensorflow to FPGA-based accelerator with a reuse-aware memory allocation for shortcut data [J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2022, 69(6): 2477-2489.
- [52]Vivek K J, Hemstad. Using the NVIDIA CUDA Stream-Ordered Memory Allocator [EB/OL]. [2021]. <https://developer.nvidia.com/blog/using-cuda-stream-ordered-memory-allocator-part-1/>
- [53]Fang W. Analysis on Dynamic Memory Allocation [J]. Computer Sciences Department, University of Wisconsin-Madison, 2013.
- [54]Peterson J L, Norman T A. Buddy systems [J]. Communications of the ACM, 1977, 20(6): 421-31.
- [55]Bonwick J. The slab allocator: An object-caching kernel memory allocator [C]. proceedings of the USENIX summer, USA: University of California Press, 1994.
- [56]Wörgötter F, Porr B. Reinforcement learning [J]. Scholarpedia, 2020, 3: 1448.
- [57]Watkins C, Dayan P. Q-learning [J]. Machine Learning, 1992, 8: 279-92.
- [58]Srikant R, Ying L. Finite-Time Error Bounds For Linear Stochastic Approximation and TD Learning [J]. arXiv: abs/1902.00923, 2019: 2803-2830.
- [59]Hasselt H V, Guez A, Silver D. Deep Reinforcement Learning with Double Q-Learning [C]. proceedings of the AAAI Conference on Artificial Intelligence, USA: AAAI, 2015.
- [60]Foerster J, Nardelli N, Farquhar G, et al. Stabilising experience replay for deep multi-agent reinforcement learning [C]. proceedings of the International conference on machine learning, USA: ACM, 2017: 1146-1155.
- [61]Lowe R, Wu Y, Tamar A, et al. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments [J]. arXiv: abs/1706.02275, 2017.

- [62] Tampuu A, Matiisen T, Kodelja D, et al. Multiagent cooperation and competition with deep reinforcement learning [J]. PLoS ONE, 2015, 12.
- [63] Omidshafiei S, Pazis J, Amato C, et al. Deep decentralized multi-task multi-agent reinforcement learning under partial observability [C]. proceedings of the International Conference on Machine Learning, USA: ACM, 2017: 2681-2690.
- [64] Simonyan K, Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition [J]. CoRR, 2014, abs/1409.1556.
- [65] Yousefzadeh-Asl-Miandoab E. Profiling with TensorFlow [EB/OL]. [2022]. <https://medium.com/mllearning-ai/profiling-with-tensorflow-9eaa283e8c3>
- [66] Weisberg S. Applied linear regression[M].USA: John Wiley & Sons, 2005.
- [67] Tao Y, Papadias D. Maintaining sliding window skylines on data streams [J]. IEEE Transactions on Knowledge and Data Engineering, 2006, 18(3): 377-91.
- [68] Howard A G, Zhu M, Chen B, et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications [J]. arXiv, 2017, abs/1704.04861.
- [69] He K, Zhang X, Ren S, Sun J. Deep Residual Learning for Image Recognition [J]. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015: 770-778.
- [70] Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the Inception Architecture for Computer Vision [J]. Computer Vision and Pattern Recognition, 2015: 2818-2826.
- [71] Huang G, Liu Z, Weinberger K Q. Densely Connected Convolutional Networks [J]. 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016: 2261-2269.